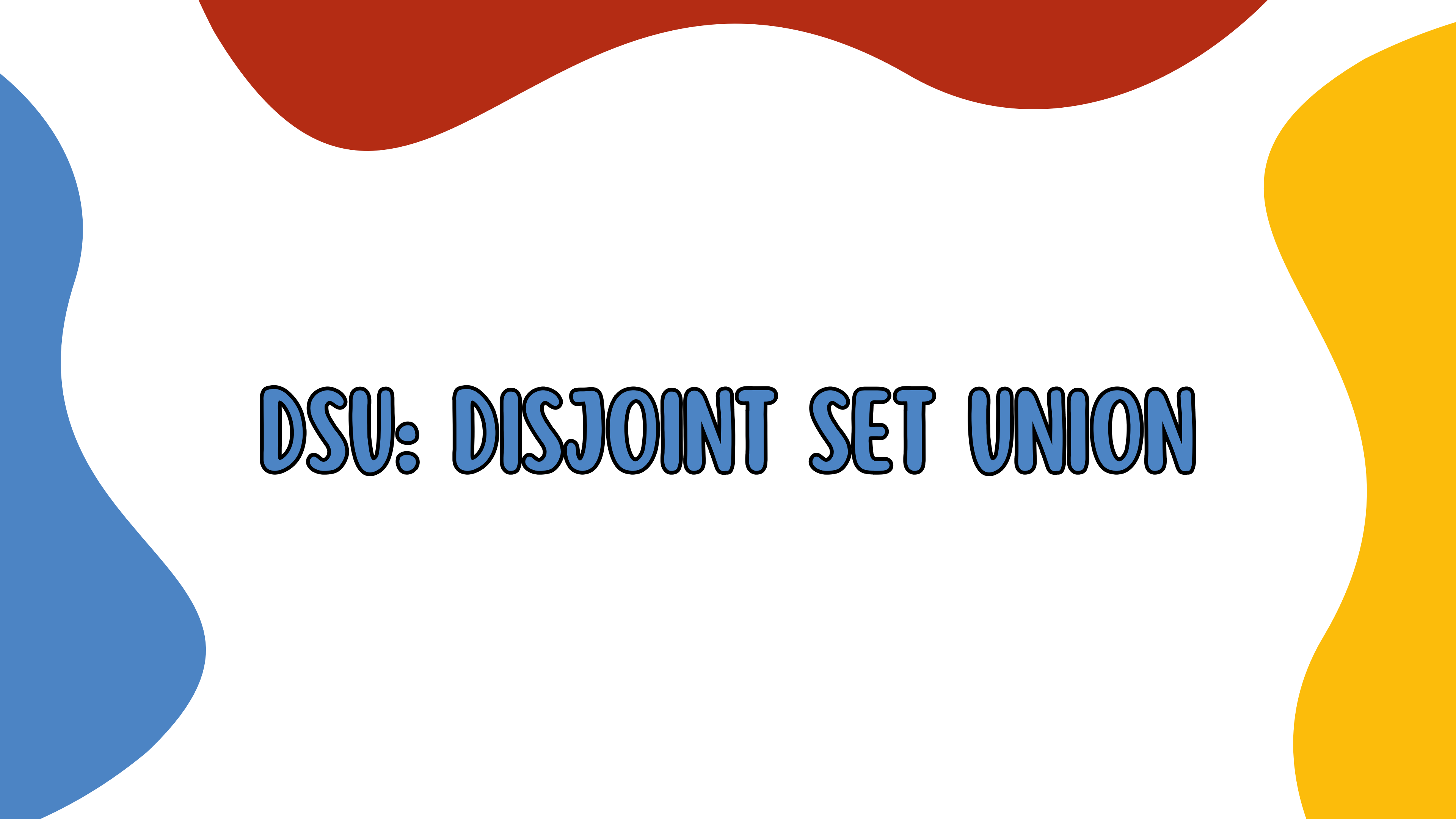




DSU: DISJOINT SET UNION

PROBLEMA

- Tenemos **N** países.
- Estos países pueden formar alianzas. El aliado de mi aliado también es mi aliado. Es posible que dos países comiencen una guerra solo si no son aliados.
- Nos van llegando consultas para dos países **u** y **v**:
 - A veces, se nos informa que **u** y **v** formaron una alianza.
 - A veces nos preguntan, ¿podrían **u** y **v** comenzar una guerra?

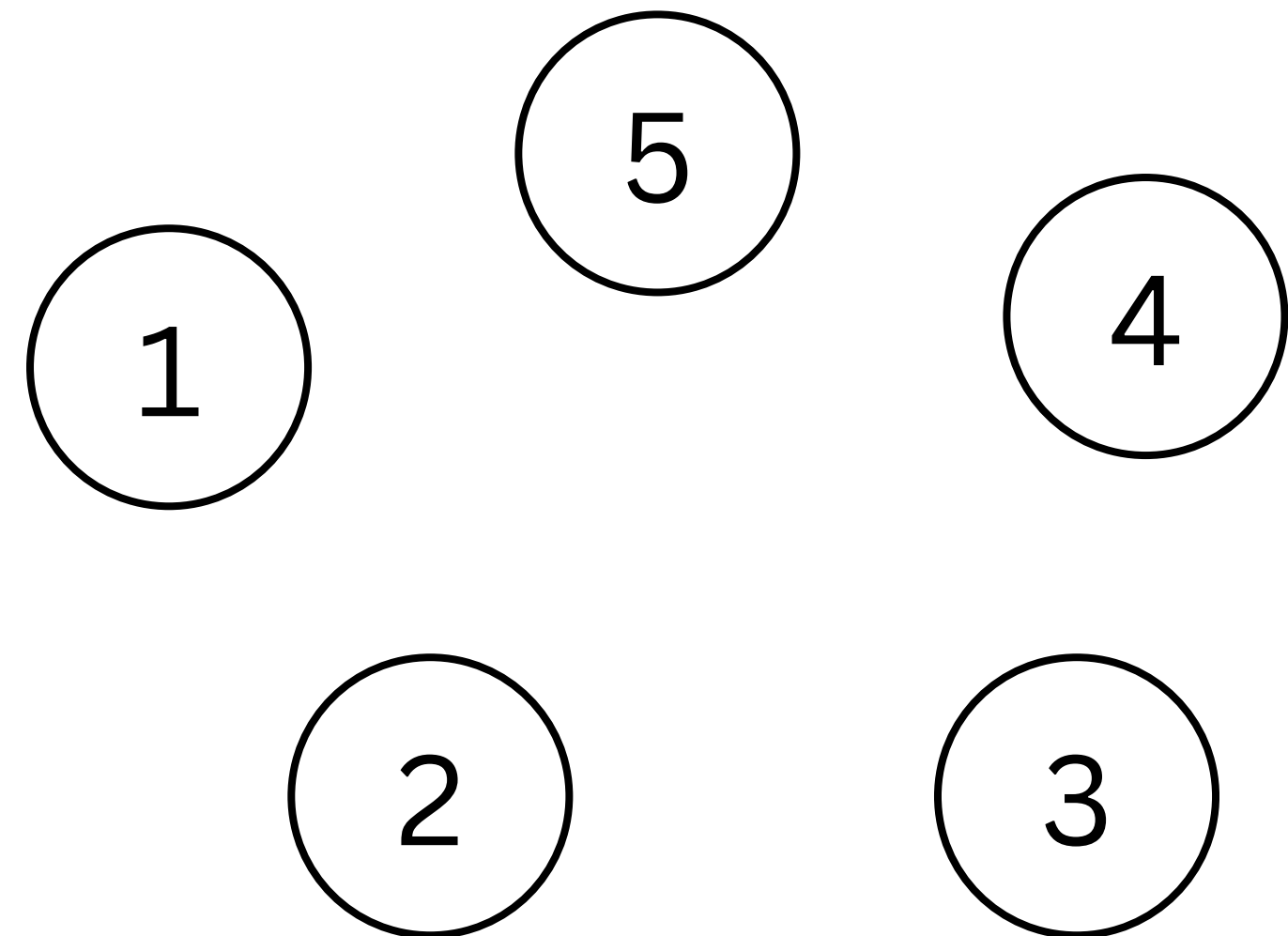
ABSTRACCIÓN

- Tenemos N países.
- Nos van llegando consultas para dos nodos u y v :
 - A veces, se nos pide agregar una arista entre u y v .
 - A veces nos preguntan, ¿existe algún camino de u hacia v ?, en otras palabras, ¿Están u y v conectados?

EJEMPLO

INPUT:

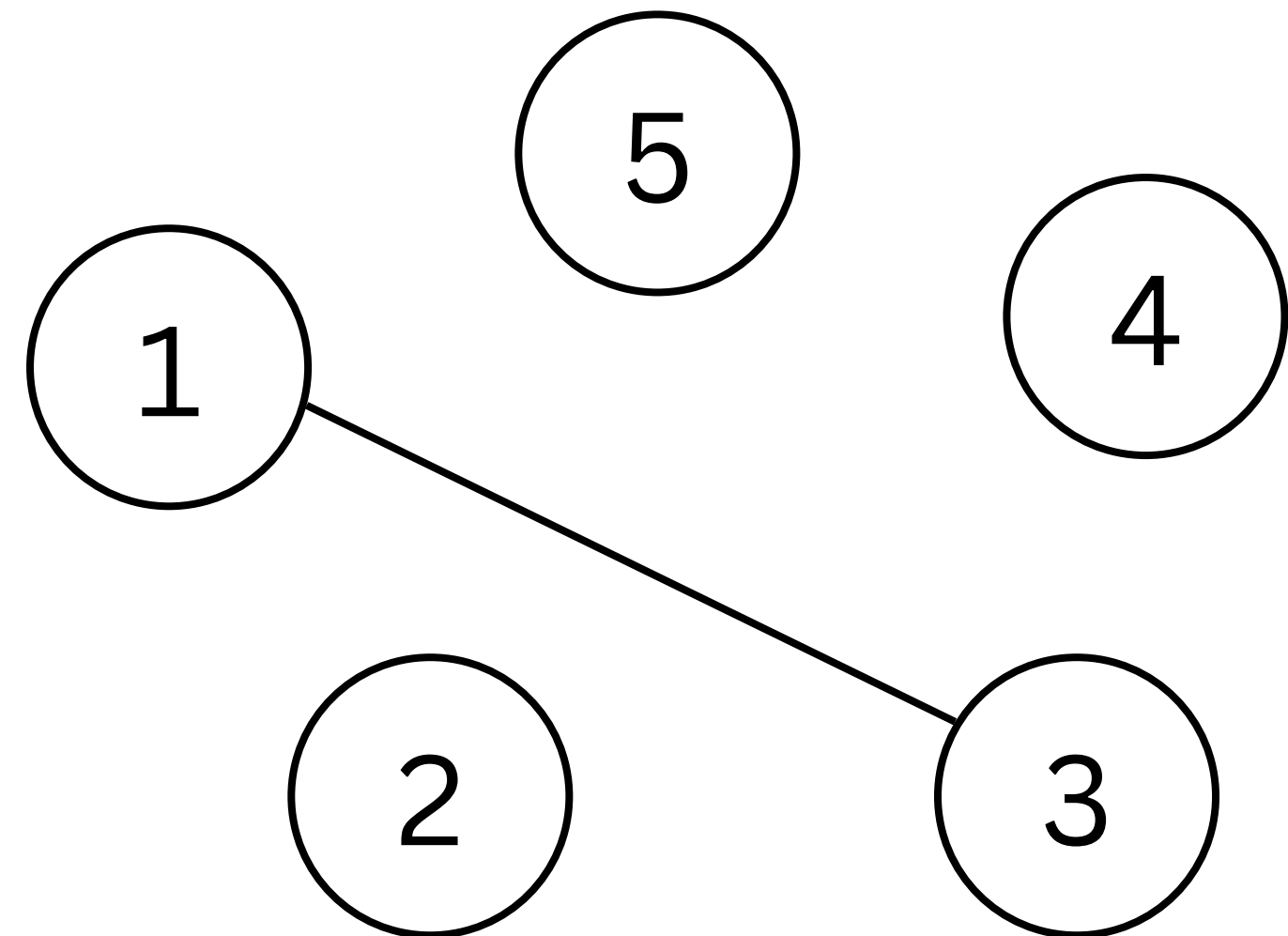
- Unir 1 y 3
- Unir 3 y 4
- ¿Están 4 y 1 conectados?
- Unir 2 y 5
- ¿Están 2 y 4 conectados?
- Unir 1 y 2
- ¿Están 2 y 4 conectados?



EJEMPLO

INPUT:

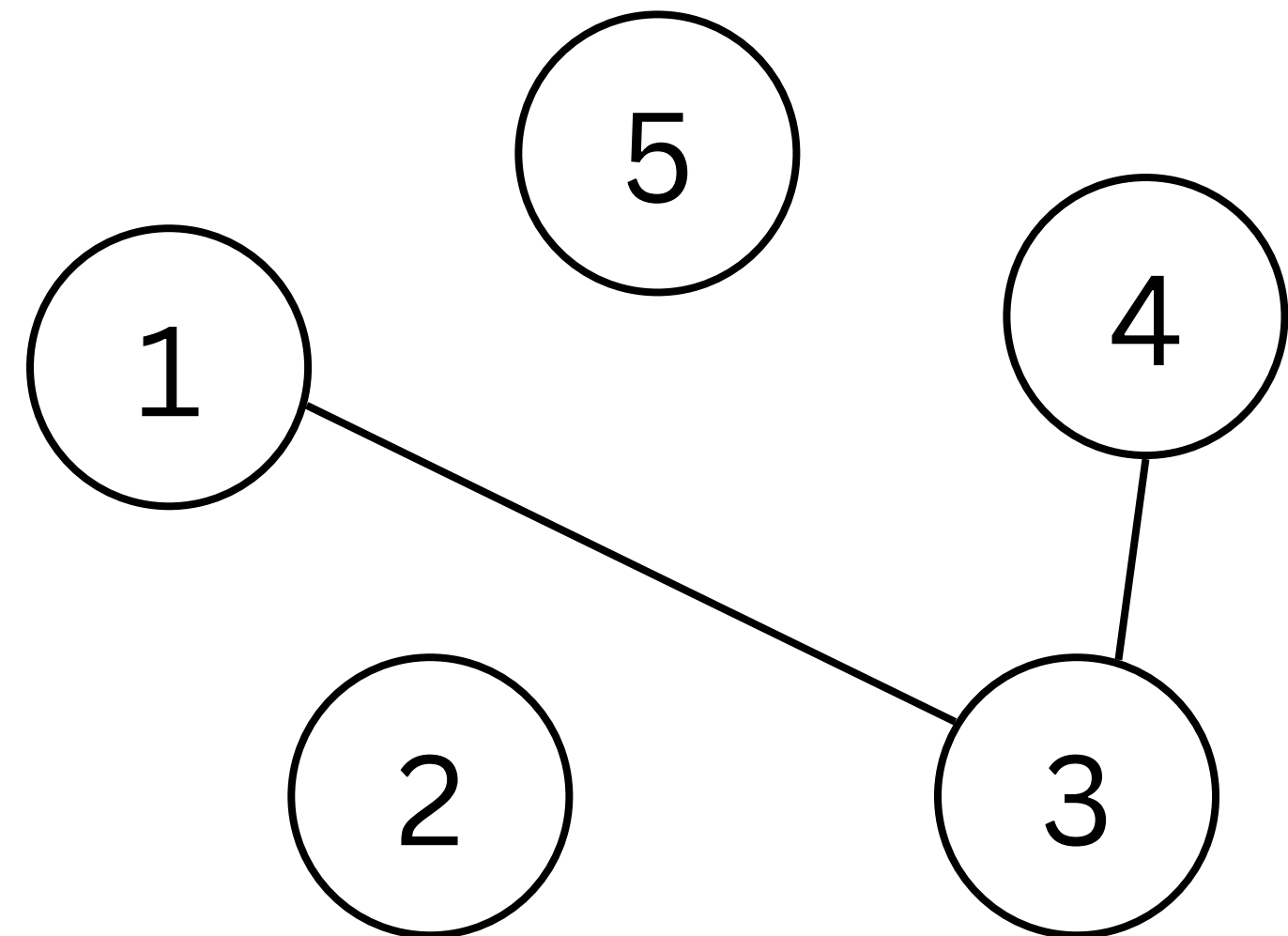
- **Unir 1 y 3**
- **Unir 3 y 4**
- **¿Están 4 y 1 conectados?**
- **Unir 2 y 5**
- **¿Están 2 y 4 conectados?**
- **Unir 1 y 2**
- **¿Están 2 y 4 conectados?**



EJEMPLO

INPUT:

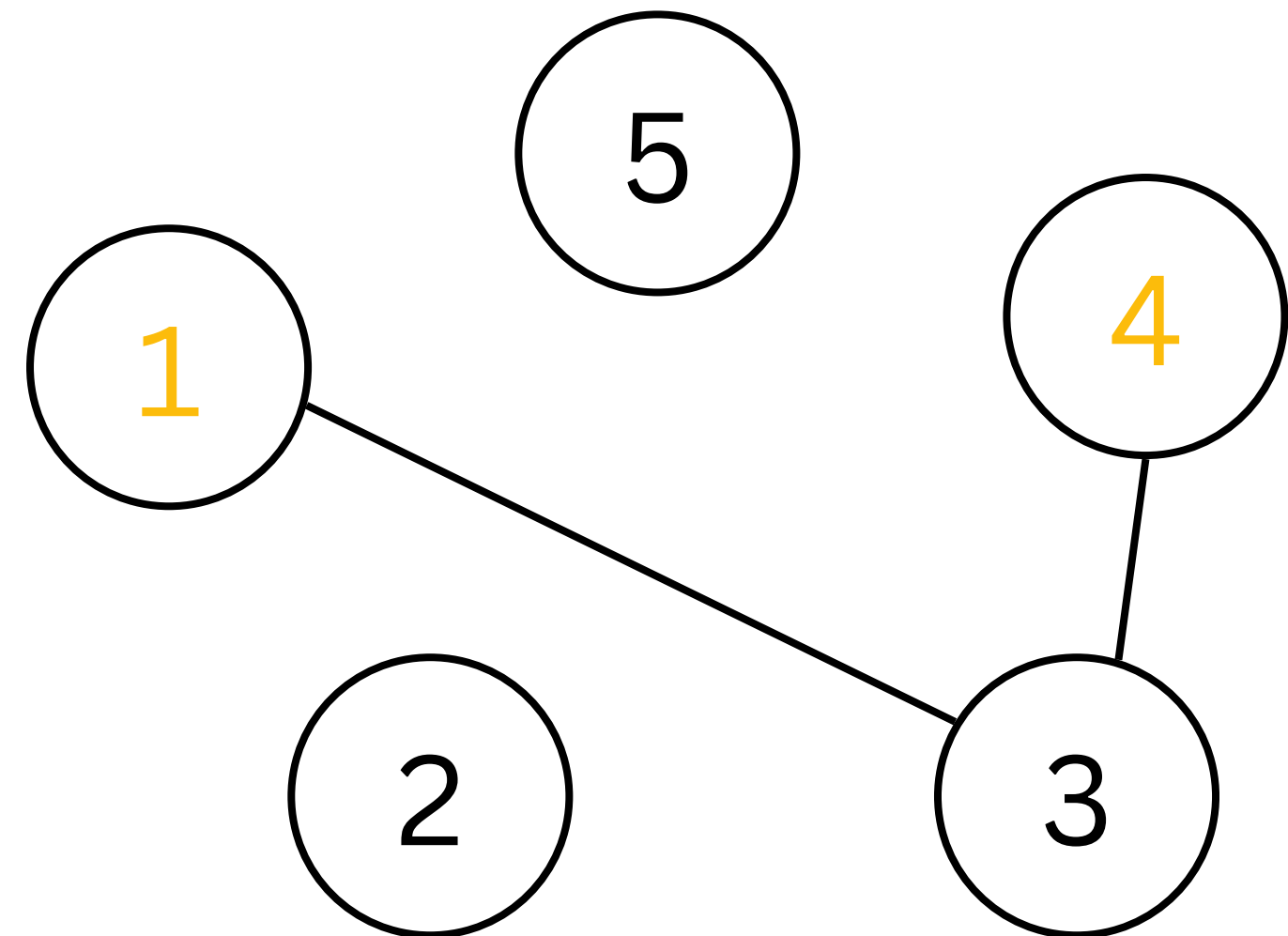
- **Unir 1 y 3**
- **Unir 3 y 4**
- ¿Están 4 y 1 conectados?
- **Unir 2 y 5**
- ¿Están 2 y 4 conectados?
- **Unir 1 y 2**
- ¿Están 2 y 4 conectados?



EJEMPLO

INPUT:

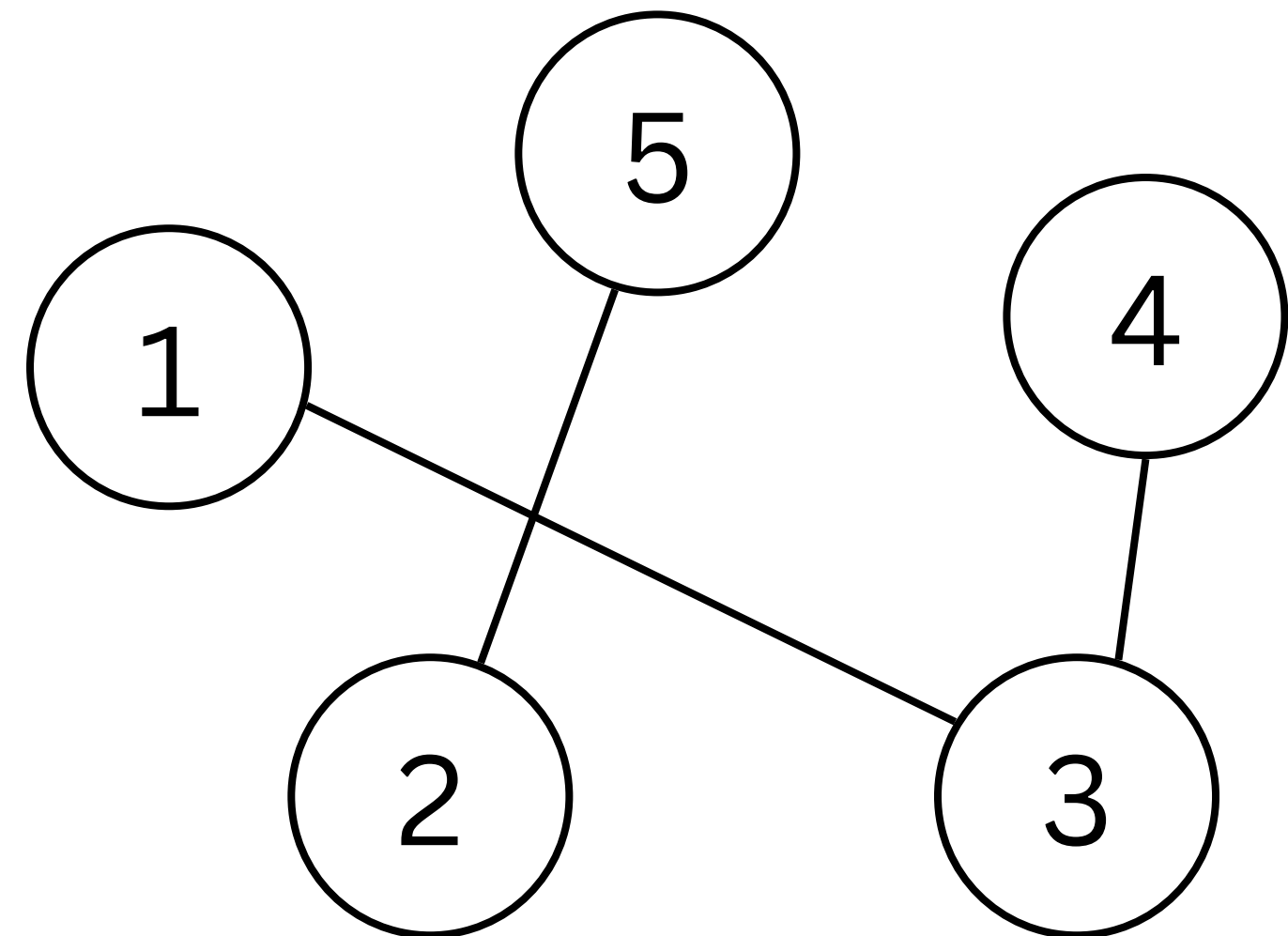
- **Unir 1 y 3**
- **Unir 3 y 4**
- ¿Están 4 y 1 conectados? **Sí**
- **Unir 2 y 5**
- ¿Están 2 y 4 conectados?
- **Unir 1 y 2**
- ¿Están 2 y 4 conectados?



EJEMPLO

INPUT:

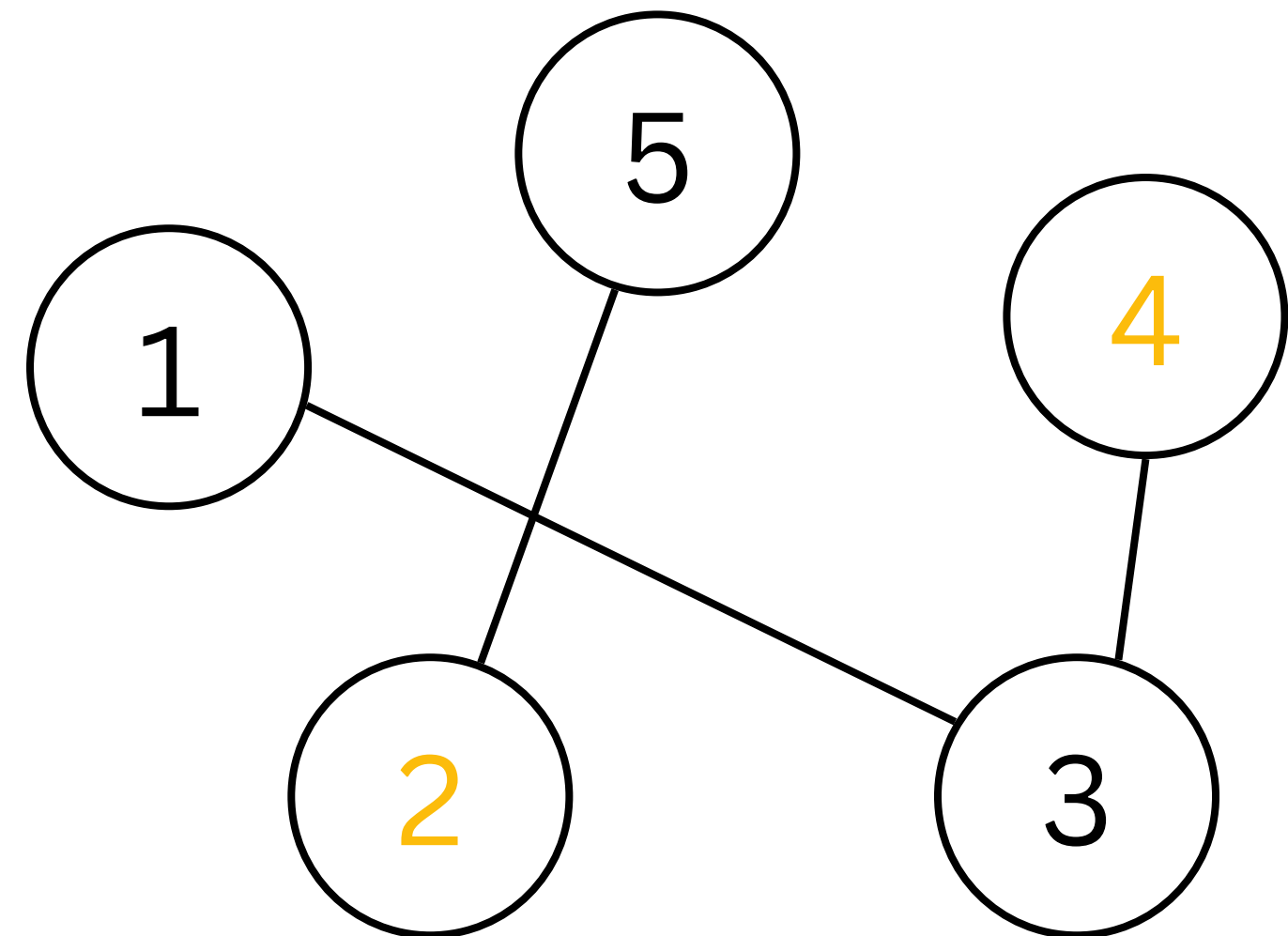
- **Unir 1 y 3**
- **Unir 3 y 4**
- ¿Están 4 y 1 conectados? **Sí**
- **Unir 2 y 5**
- ¿Están 2 y 4 conectados?
- **Unir 1 y 2**
- ¿Están 2 y 4 conectados?



EJEMPLO

INPUT:

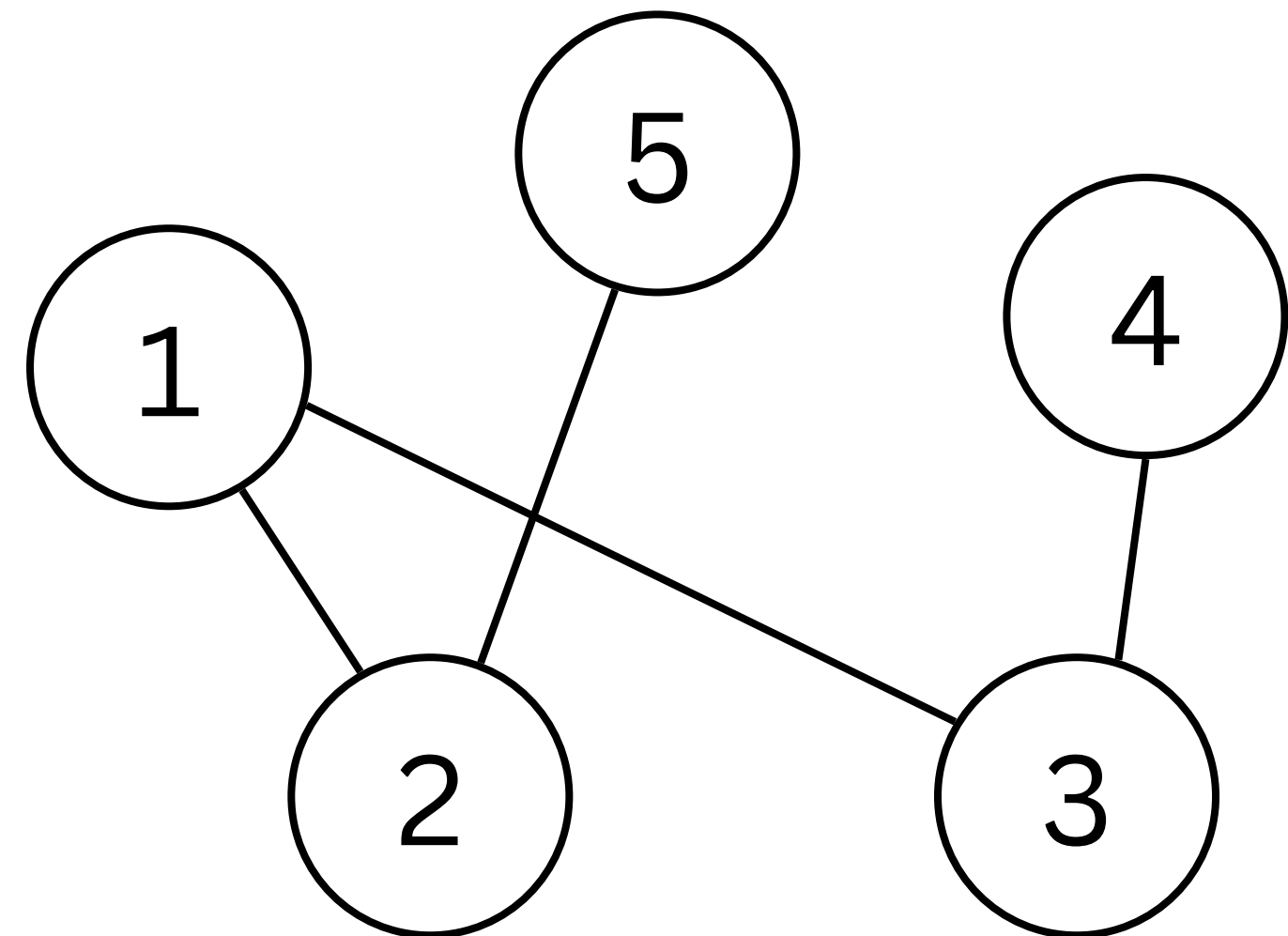
- **Unir 1 y 3**
- **Unir 3 y 4**
- ¿Están 4 y 1 conectados? **Sí**
- **Unir 2 y 5**
- ¿Están 2 y 4 conectados? **No**
- **Unir 1 y 2**
- ¿Están 2 y 4 conectados?



EJEMPLO

INPUT:

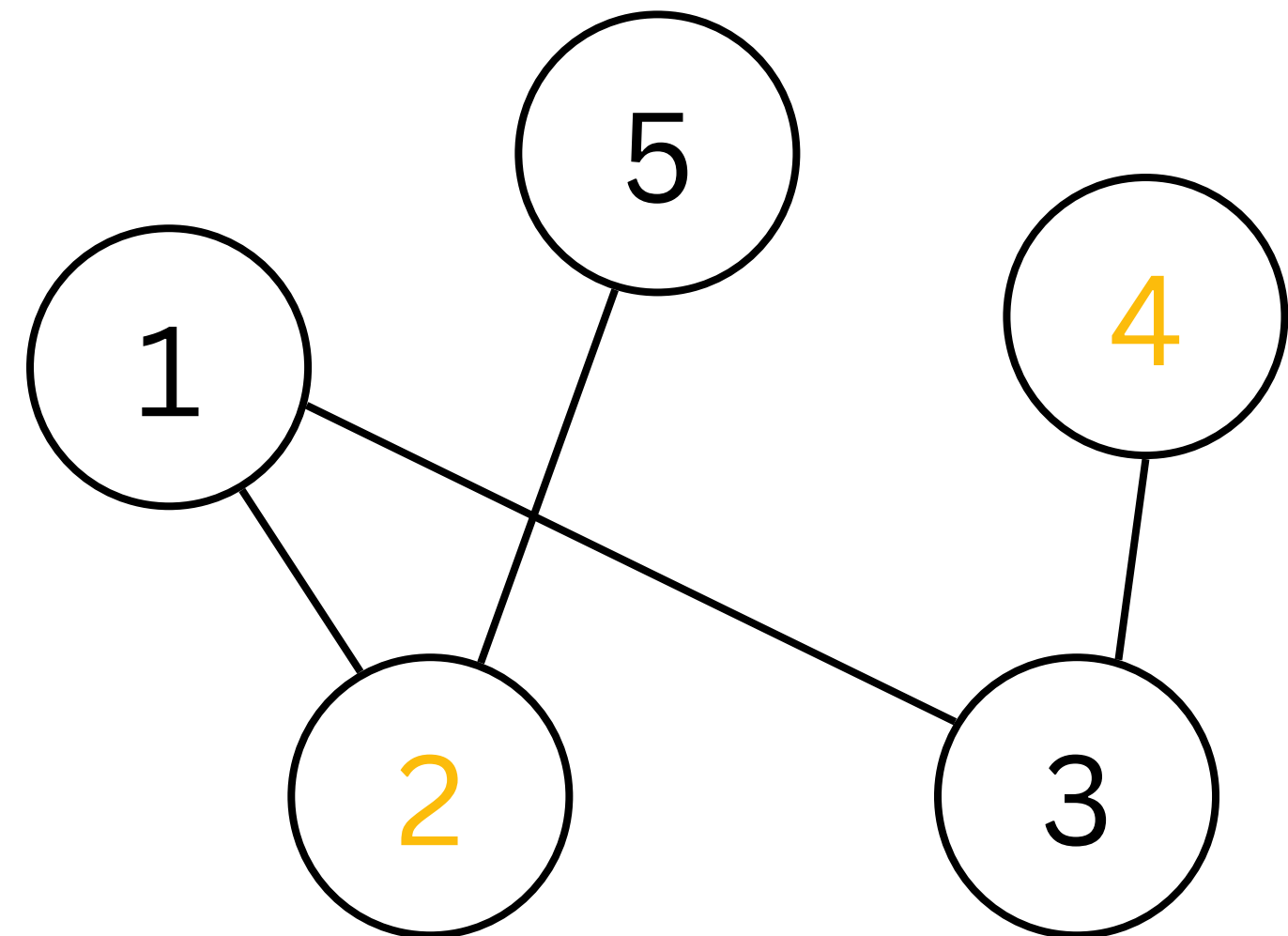
- **Unir 1 y 3**
- **Unir 3 y 4**
- ¿Están 4 y 1 conectados? **Sí**
- **Unir 2 y 5**
- ¿Están 2 y 4 conectados? **No**
- **Unir 1 y 2**
- ¿Están 2 y 4 conectados?



EJEMPLO

INPUT:

- **Unir 1 y 3**
- **Unir 3 y 4**
- ¿Están 4 y 1 conectados? Sí
- **Unir 2 y 5**
- ¿Están 2 y 4 conectados? No
- **Unir 1 y 2**
- ¿Están 2 y 4 conectados? Sí



SOLUCIÓN INGENUA

Ingenua: Hacer exactamente lo que nos piden.

- **Crear un grafo de N nodos inicialmente vacío (eg. lista de adyacencia).**
- **Al unir dos nodos, agregar una arista al grafo.**
- **Al consultar sobre la conectividad de dos nodos, buscar un camino entre ambos (eg. con BFS/DFS/Dijkstra).**

COMPLEJIDAD INGENUA

Ingenua: Hacer exactamente lo que nos piden.

- Crear un grafo de N nodos inicialmente vacío: $O(N)$
- Al unir dos nodos, agregar una arista al grafo: $O(1)$ por operación
- Al consultar sobre la conectividad de dos nodos, buscar un camino entre ambos: $O(|aristas| + N)$ por operación
 - El grafo nunca tendrá mas de $|operaciones|$ aristas, por ende si llamamos Q a la cantidad de consultas: $O(Q + N)$ por operación

COMPLEJIDAD INGENUA

Ingenua: Hacer exactamente lo que nos piden.

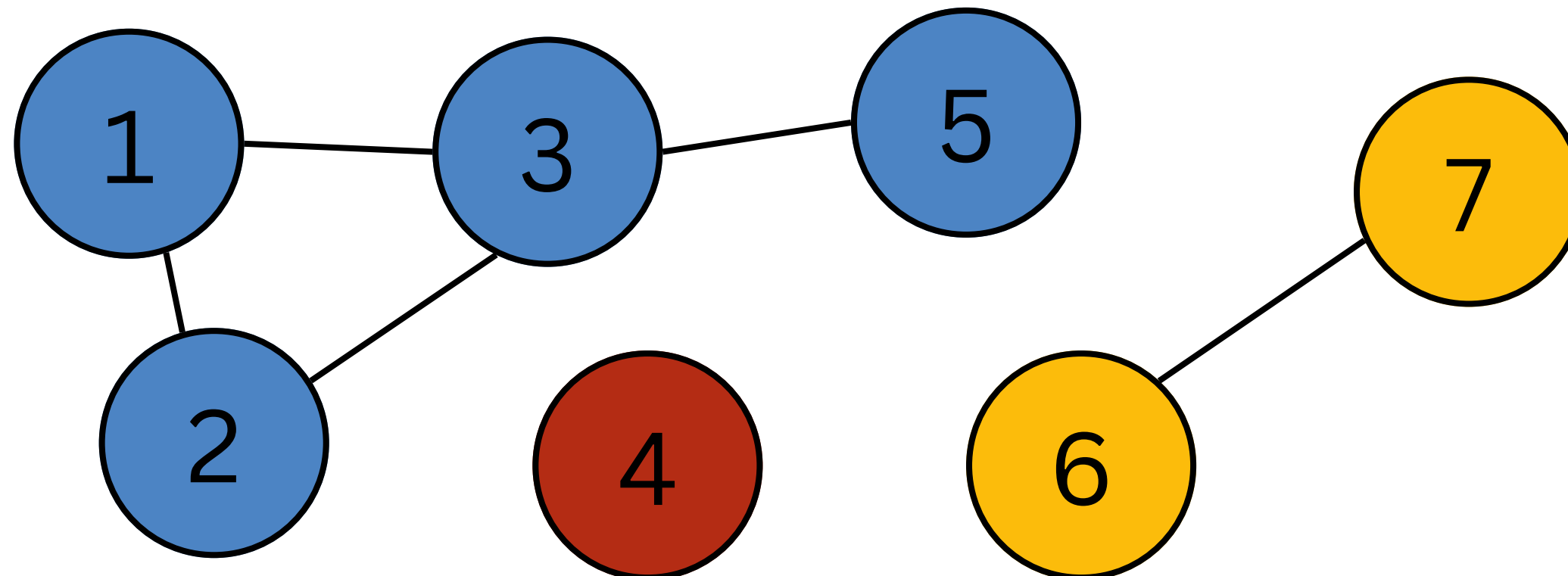
- Crear un grafo de N nodos inicialmente vacío: $O(N)$
- Al unir dos nodos, agregar una arista al grafo: $O(1)$ por operación
- Al consultar sobre la conectividad de dos nodos, buscar un camino entre ambos: $O(|aristas| + N)$ por operación
 - El grafo nunca tendrá mas de $|operaciones|$ aristas, por ende si llamamos Q a la cantidad de consultas: $O(Q + N)$ por operación
- Complejidad final: $O(Q * (Q + N))$

OBSERVACIÓN

Veamos las **componentes conexas** del grafo.

Componente conexas: Grupo de nodos conectados entre sí por aristas.

u y **v** están conectados si y sólo si están en la misma componente conexas.



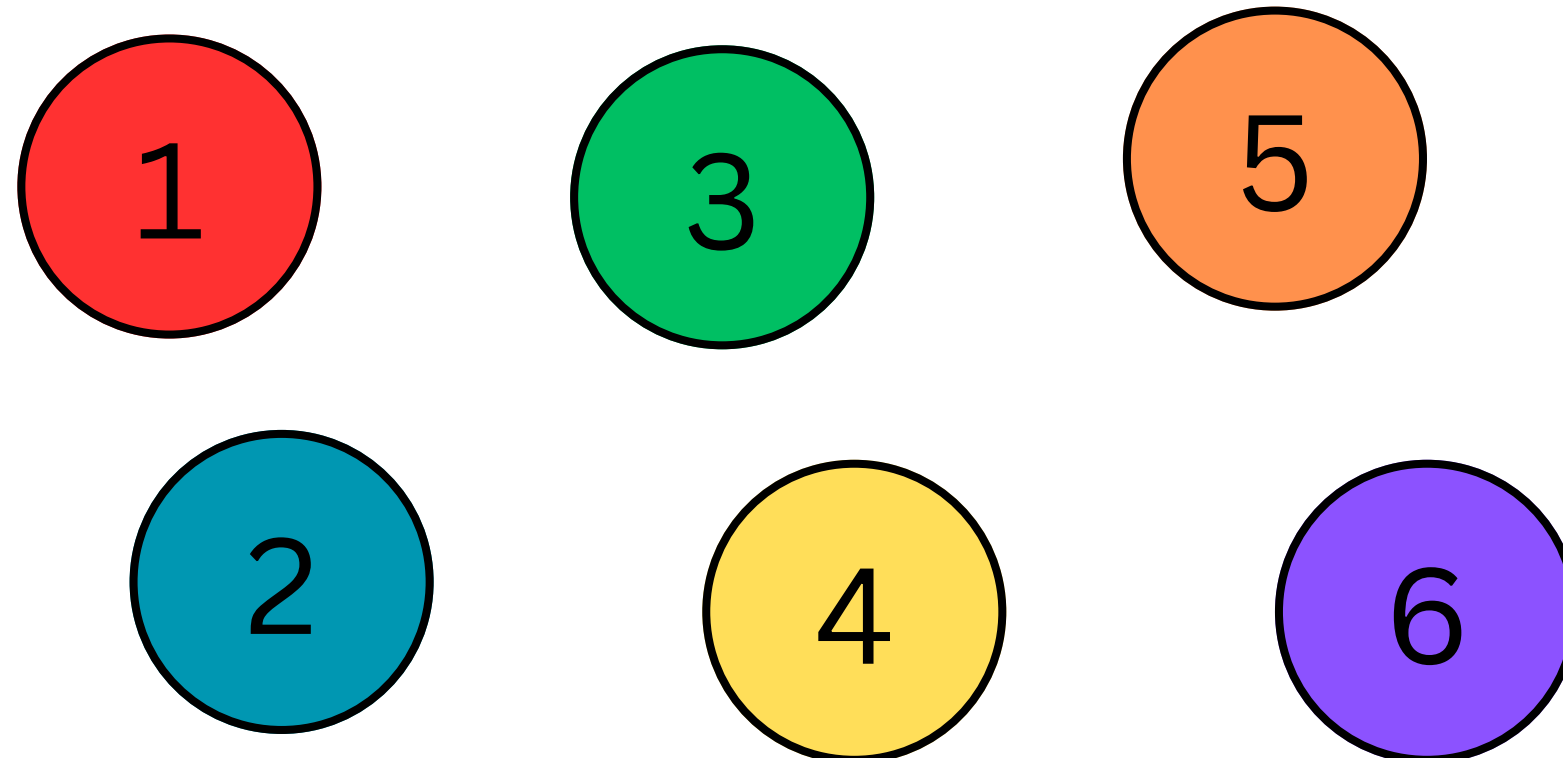
SOLUCIÓN 2.0

En lugar de mantener el grafo, mantengamos solo las componentes conexas (ie. los **colores**).

- Inicialmente, cada nodo tiene su propio color.
- Al unir dos nodos, si son de colores distintos, hay que reemplazar uno de los colores por el otro.

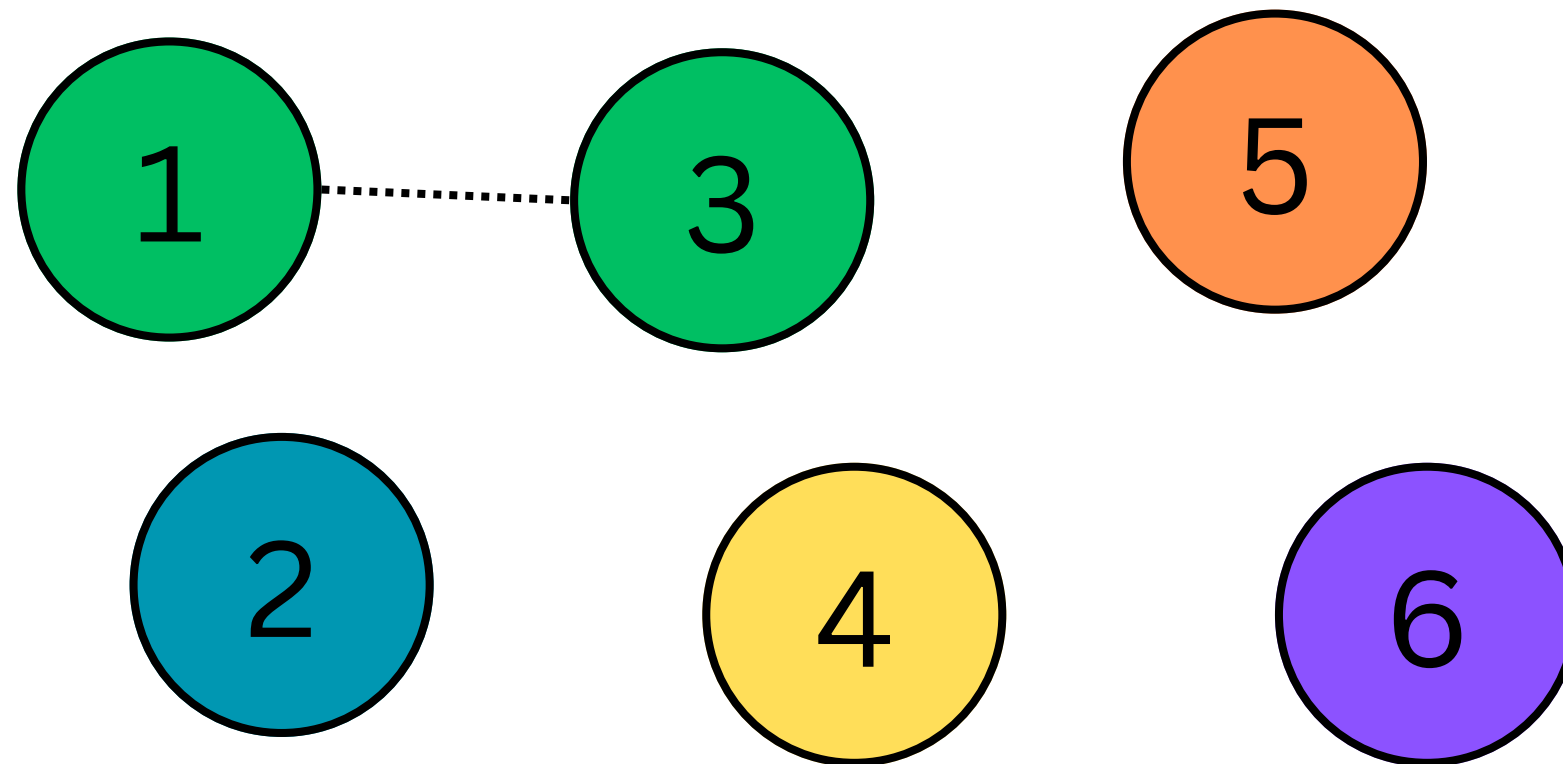
SOLUCIÓN 2.0

- Inicialmente, cada nodo tiene su propio color.
- Unir **u** y **v**: Reemplazar color de **u** por el color de **v** en el grafo.



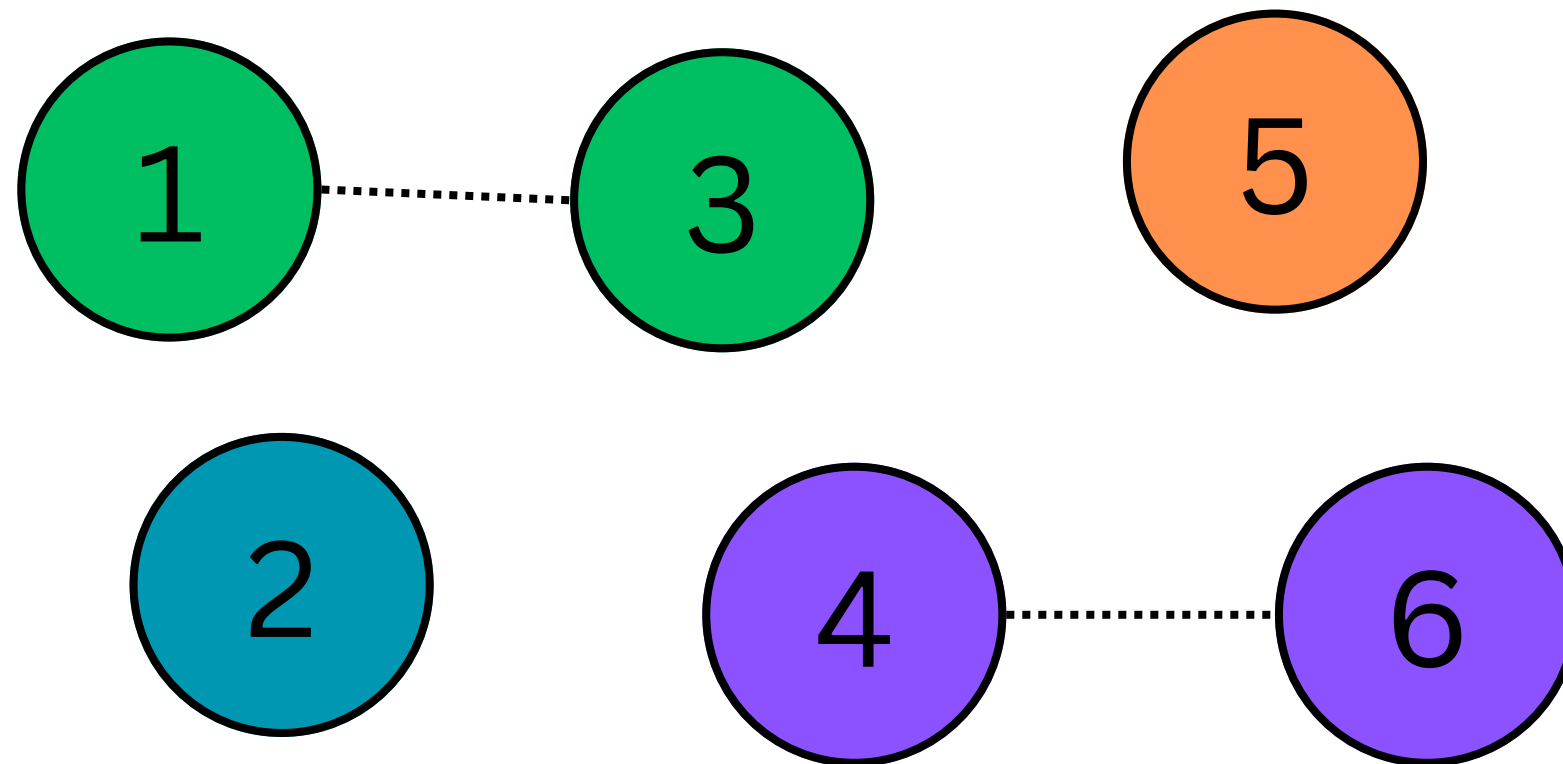
SOLUCIÓN 2.0

- Inicialmente, cada nodo tiene su propio color.
- Unir **u** y **v**: Reemplazar color de **u** por el color de **v** en el grafo.



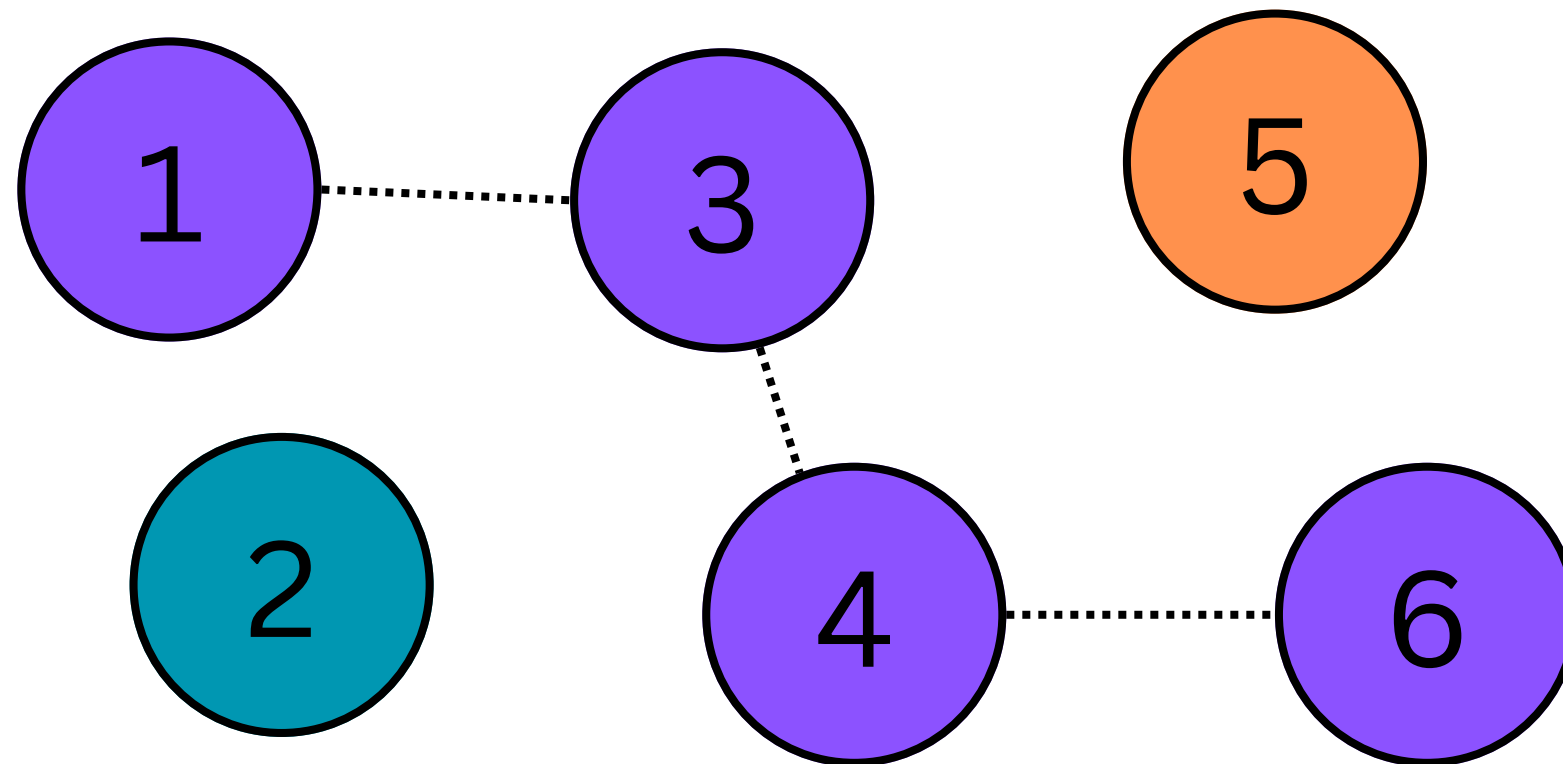
SOLUCIÓN 2.0

- Inicialmente, cada nodo tiene su propio color.
- Unir **u** y **v**: Reemplazar color de **u** por el color de **v** en el grafo.

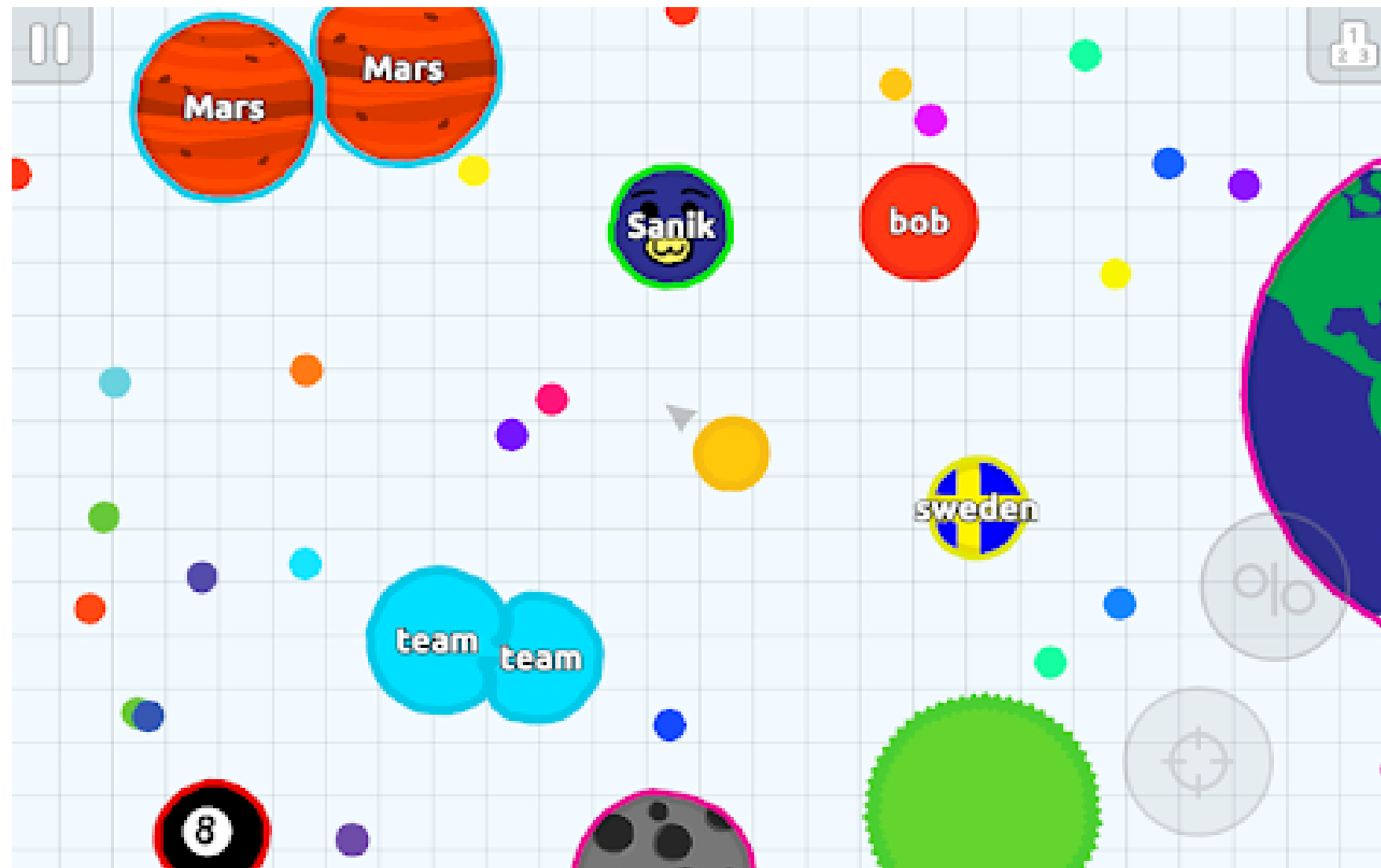


SOLUCIÓN 2.0

- Inicialmente, cada nodo tiene su propio color.
- Unir **u** y **v**: Reemplazar color de **u** por el color de **v** en el grafo.



SOLUCIÓN 2.0



SOLUCIÓN 2.0

```
color = [u for u in range(N)] # color[u] = u
for q in queries:
    if q.type == "chequear":
        print(color[q.u] == color[q.v])
    if q.type == "unir":
        color1 = color[u]
        color2 = color[v]
        for u in range(N):
            if color[u] == color1:
                color[u] = color2
```

COMPLEJIDAD 2.0

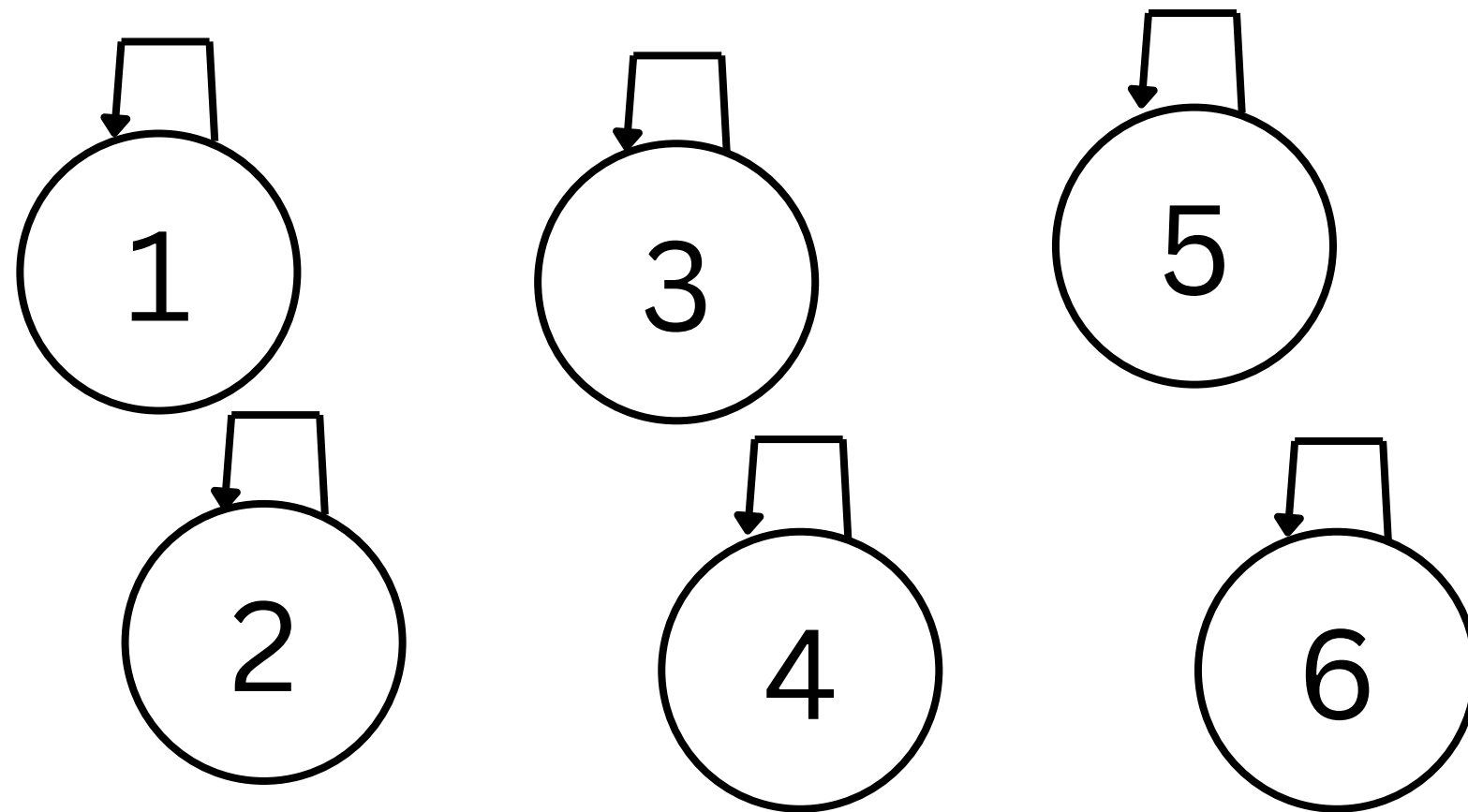
Las queries de unir toman tiempo $O(N)$ (`for u in range(N)`).

Complejidad: $O(Q * N)$

No es mucho mejor... Veamos como cambiar eso

REINTERPRETACIÓN 2.0

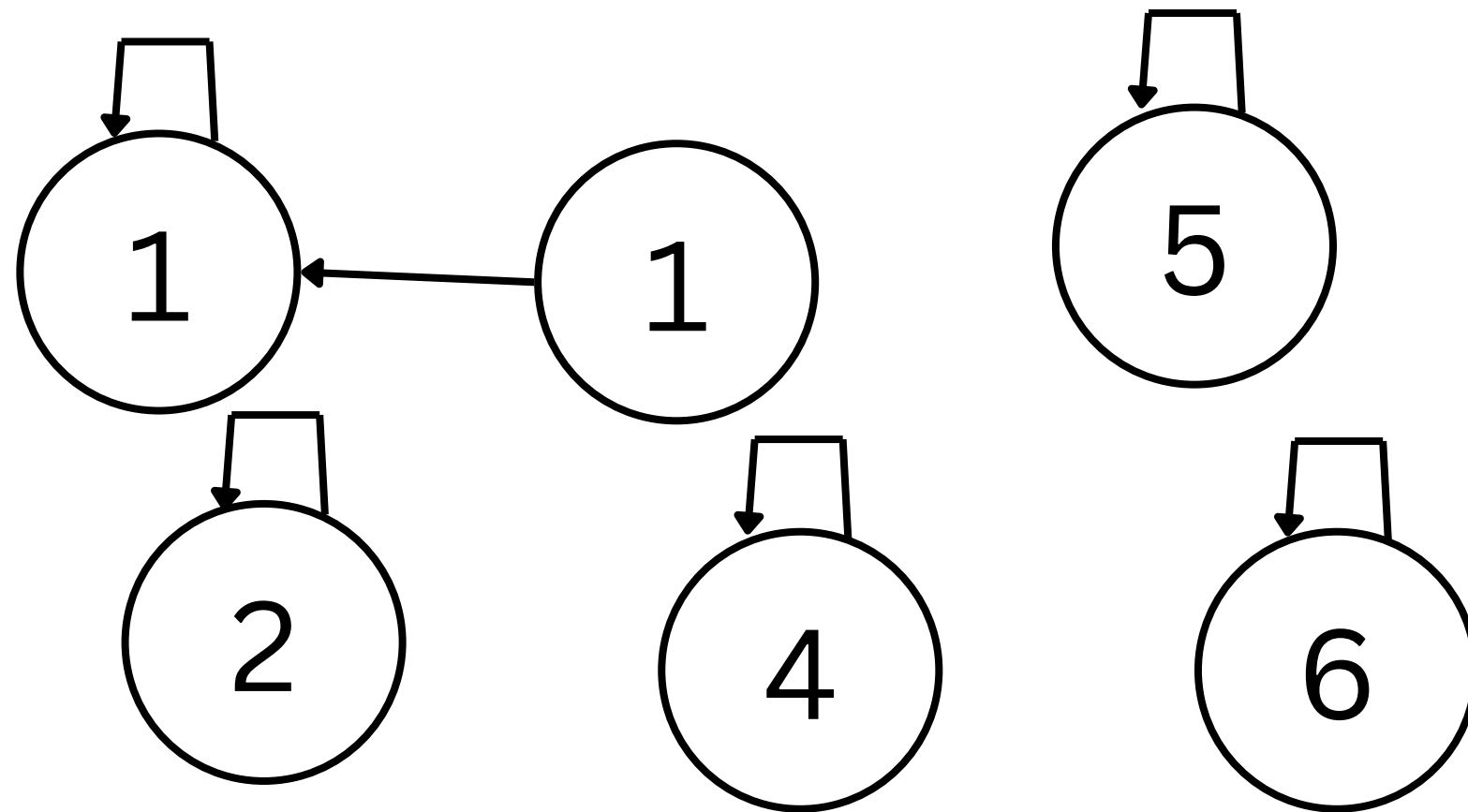
Hay **N** nodos, y **N** colores. Interpretamos cada color como un nodo!
Olvidemos los colores y cambiémoslos por flechitas:



REINTERPRETACIÓN 2.0

Hay **N** nodos, y **N** colores. Interpretamos cada color como un nodo!

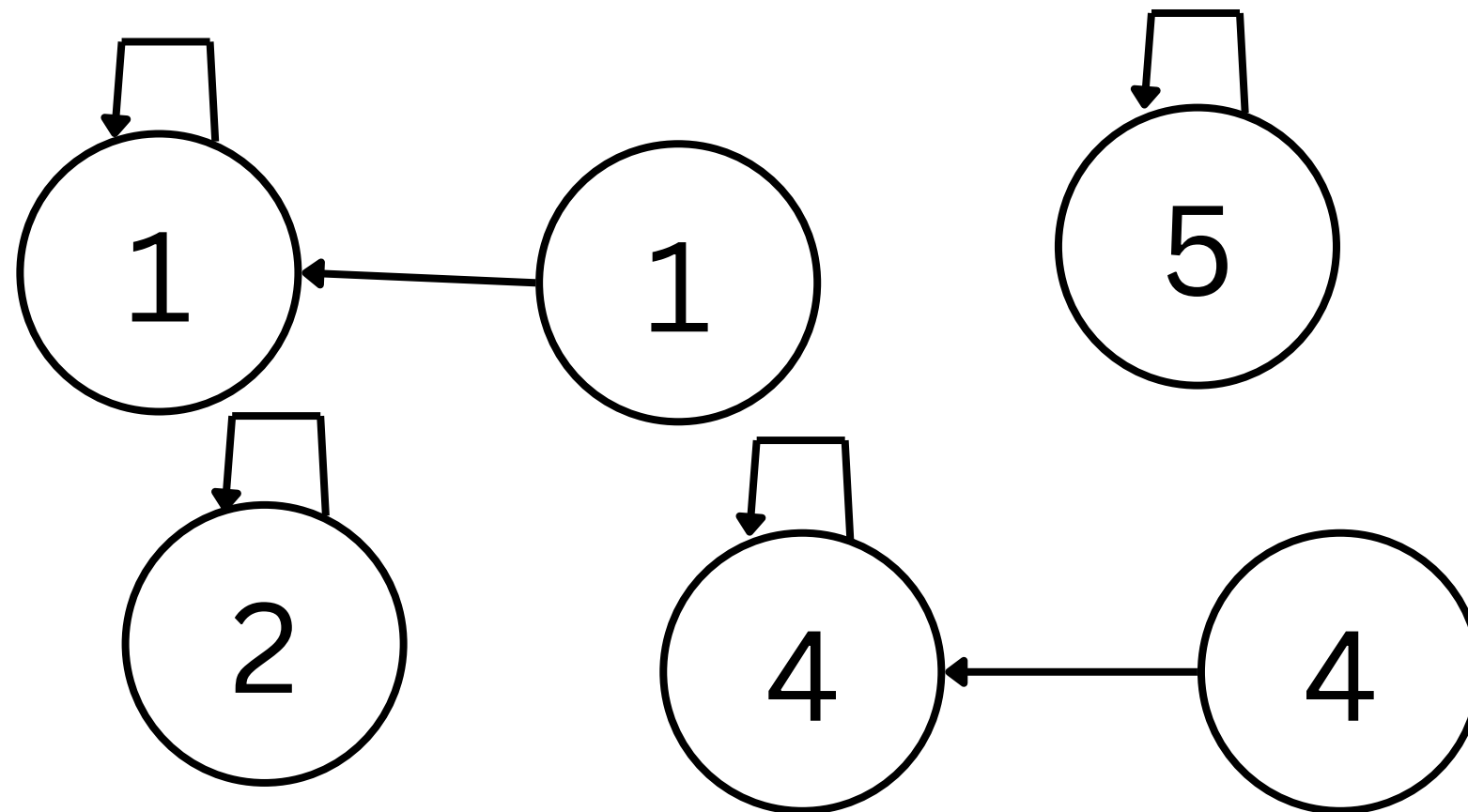
Conectar 1 y 3



REINTERPRETACIÓN 2.0

Hay **N** nodos, y **N** colores. Interpretamos cada color como un nodo!

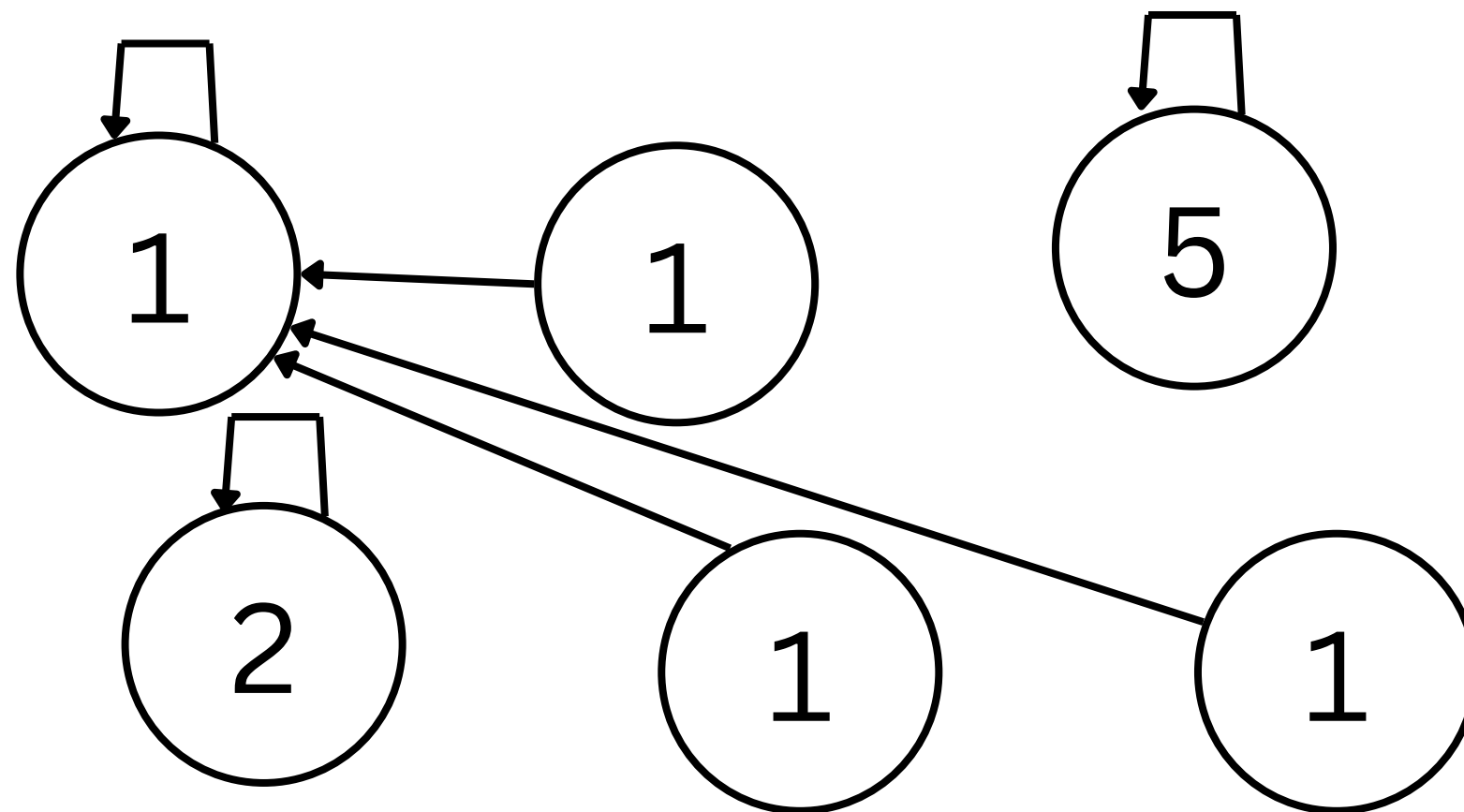
Conectar 4 y 6



REINTERPRETACIÓN 2.0

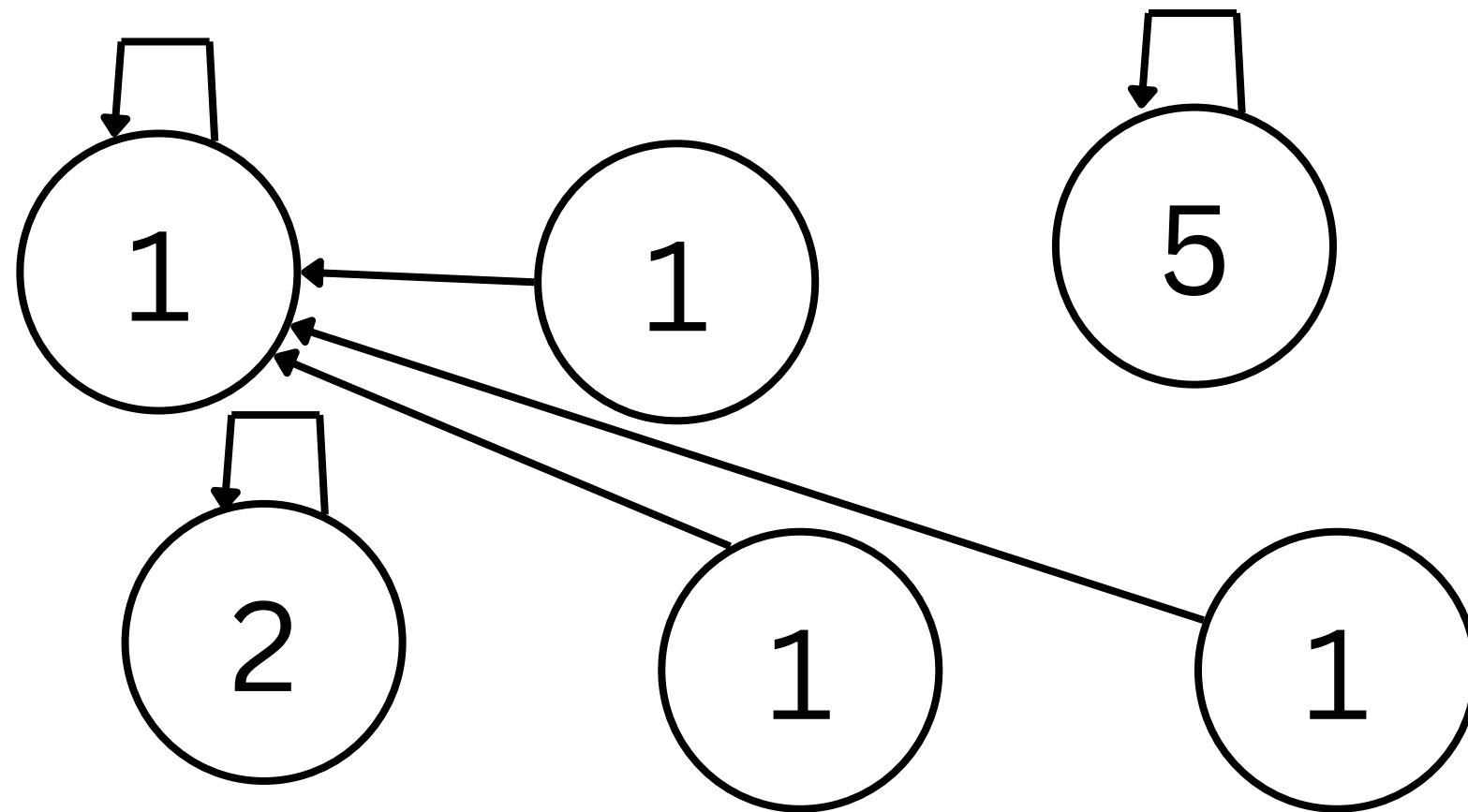
Hay **N** nodos, y **N** colores. Interpretamos cada color como un nodo!

Conectar 3 y 4



REINTERPRETACIÓN 2.0

Antes, dos nodos eran de la misma componente conexa si y sólo si **tenían el mismo color**.
Ahora, dos nodos son de la misma componente conexa si y sólo si **tienen el mismo "líder"**.

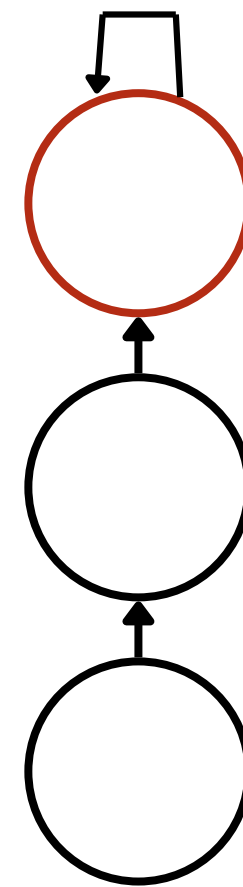
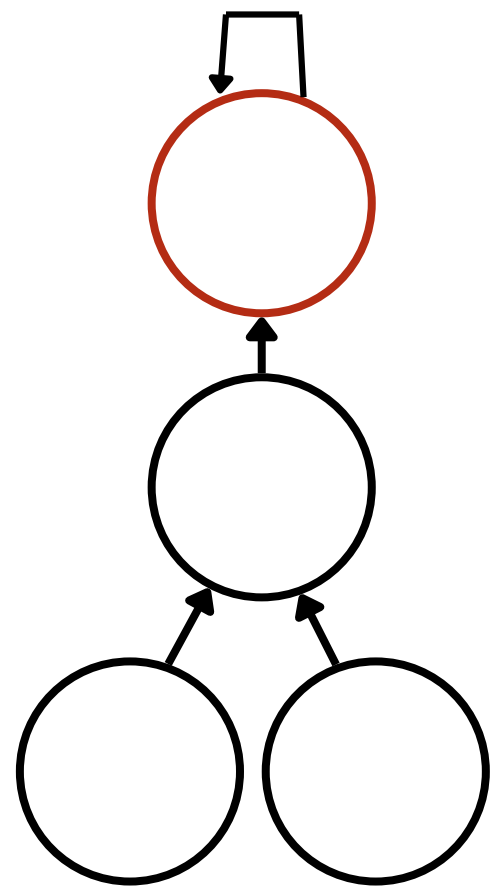


SOLUCIÓN 3.0

Nuestra operación “unir” es súper lenta, porque tenemos que modificar muchos valores (posiblemente hasta **N**).

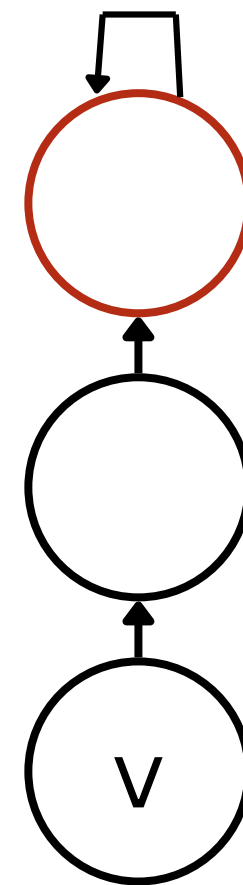
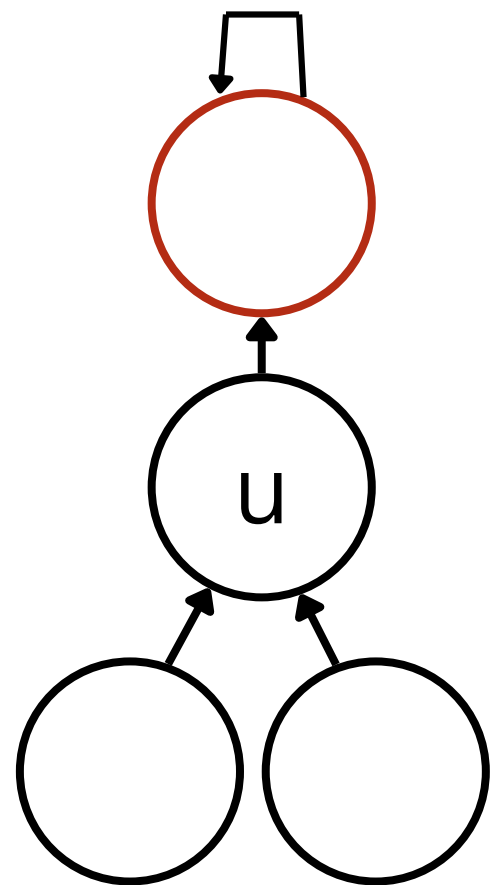
SOLUCIÓN 3.0

En lugar de mantener el **"líder"** de cada nodo, mantengamos sólo su "jefe" directo. Definimos el **"líder"** de un nodo como el jefe de su jefe de su jefe de su ... de su jefe.



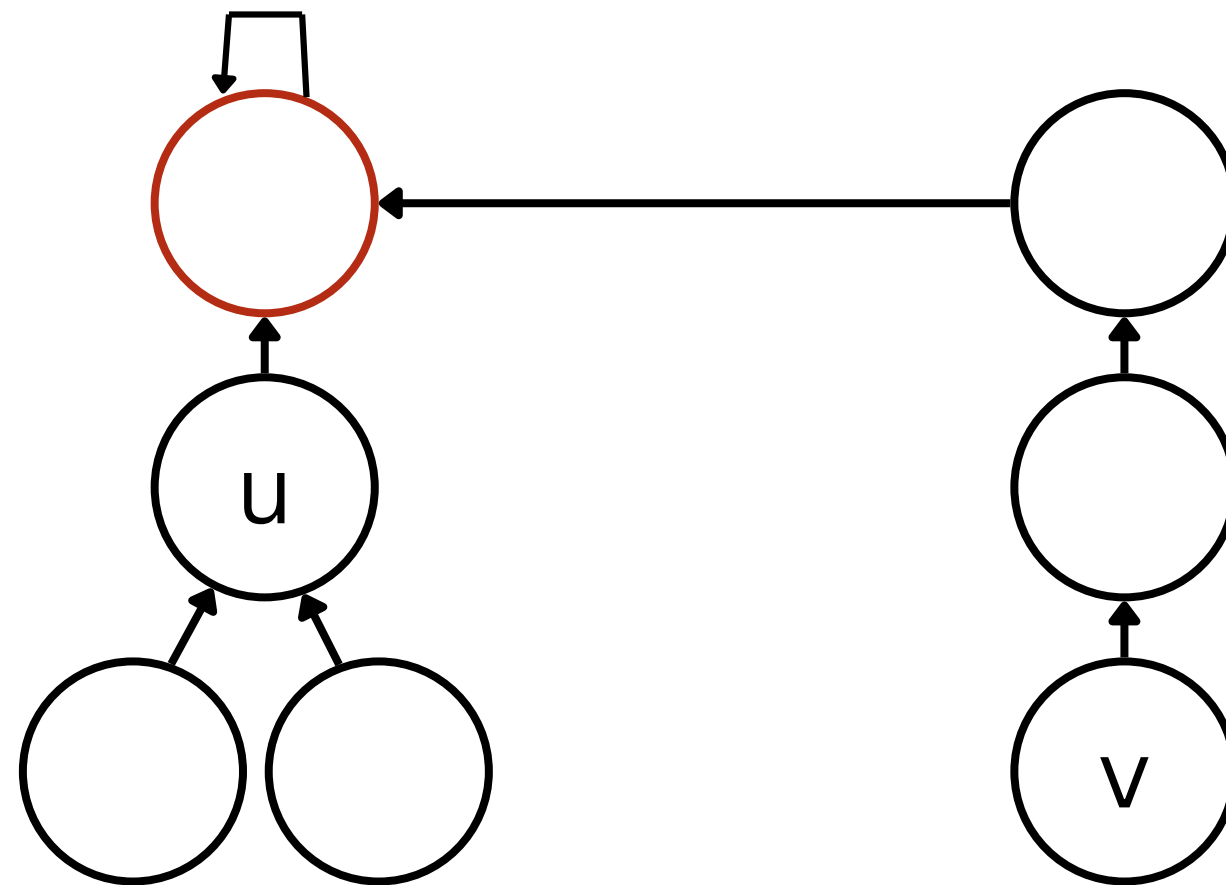
SOLUCIÓN 3.0

Para chequear si dos nodos están conectados: vemos si tienen el mismo líder.



SOLUCIÓN 3.0

Para conectar dos nodos con líderes distintos: Conectamos un líder al otro.



SOLUCIÓN 3.0

```
jefe = [u for u in range(N)]
```

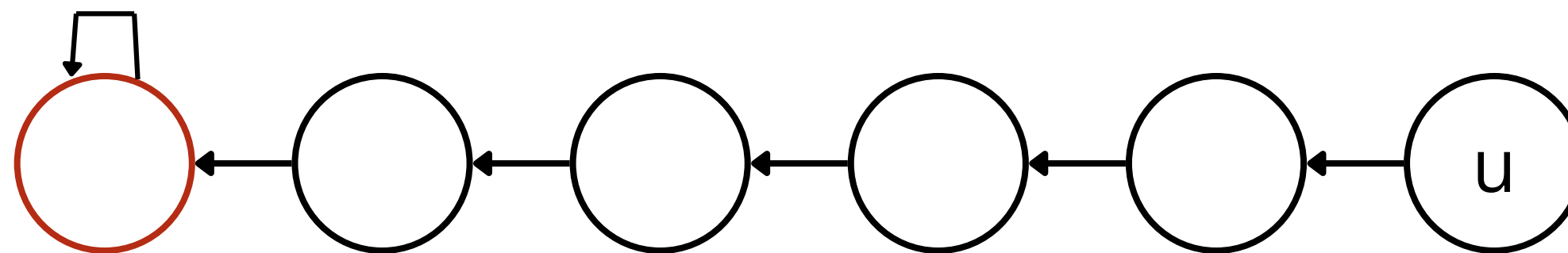
```
def lider(u):  
    if jefe[u] == u: return u  
    return lider(jefe[u])
```

```
def unir(u, v):  
    u = lider(u)  
    v = lider(v)  
    if u != v: jefe[v] = u
```

COMPLEJIDAD 3.0

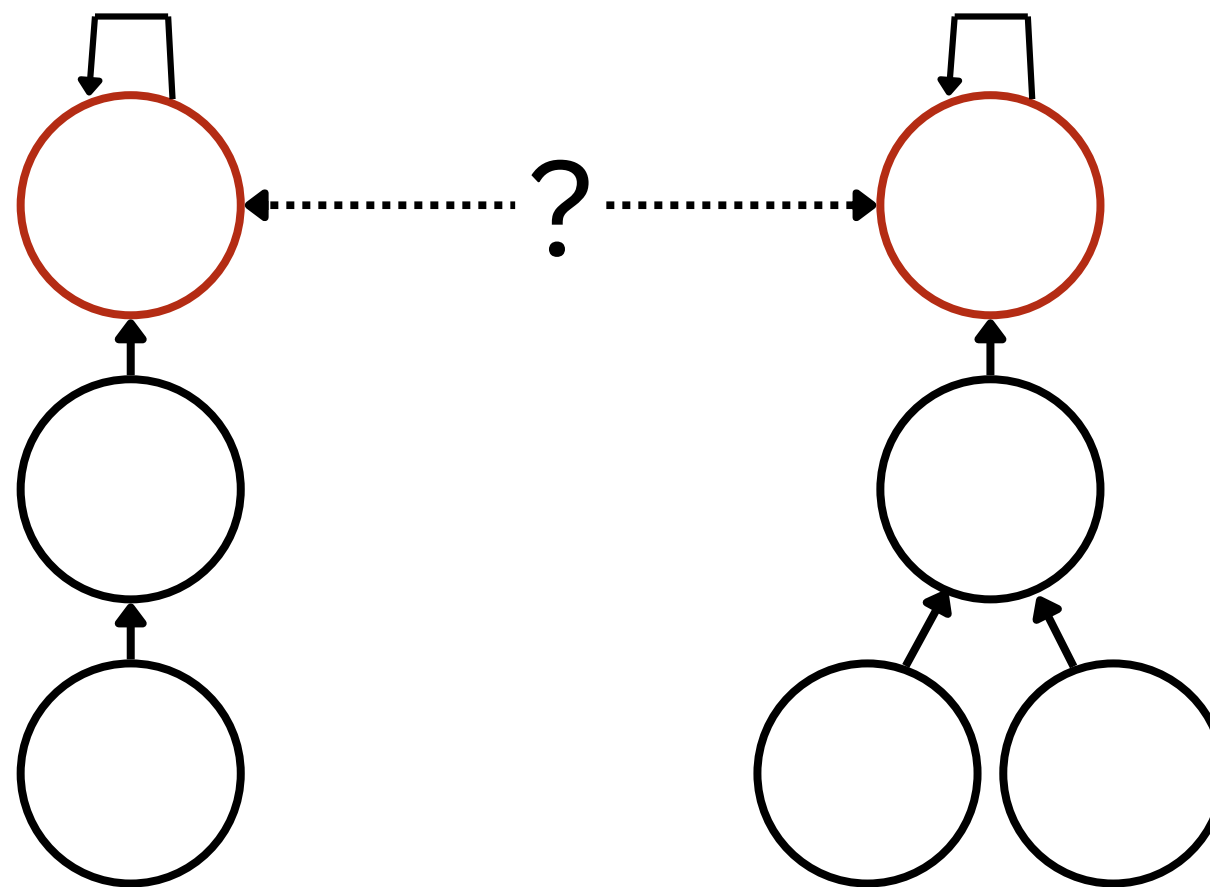
¿Cuál es la complejidad de la función $\text{lider}(u)$?

¡Podría ser $O(N)$!



SOLUCIÓN 3.1

Por suerte hay un cambio pequeño pero poderoso que podemos hacer:
Al unir dos líderes, ¿cuál debieramos conectar a cuál?

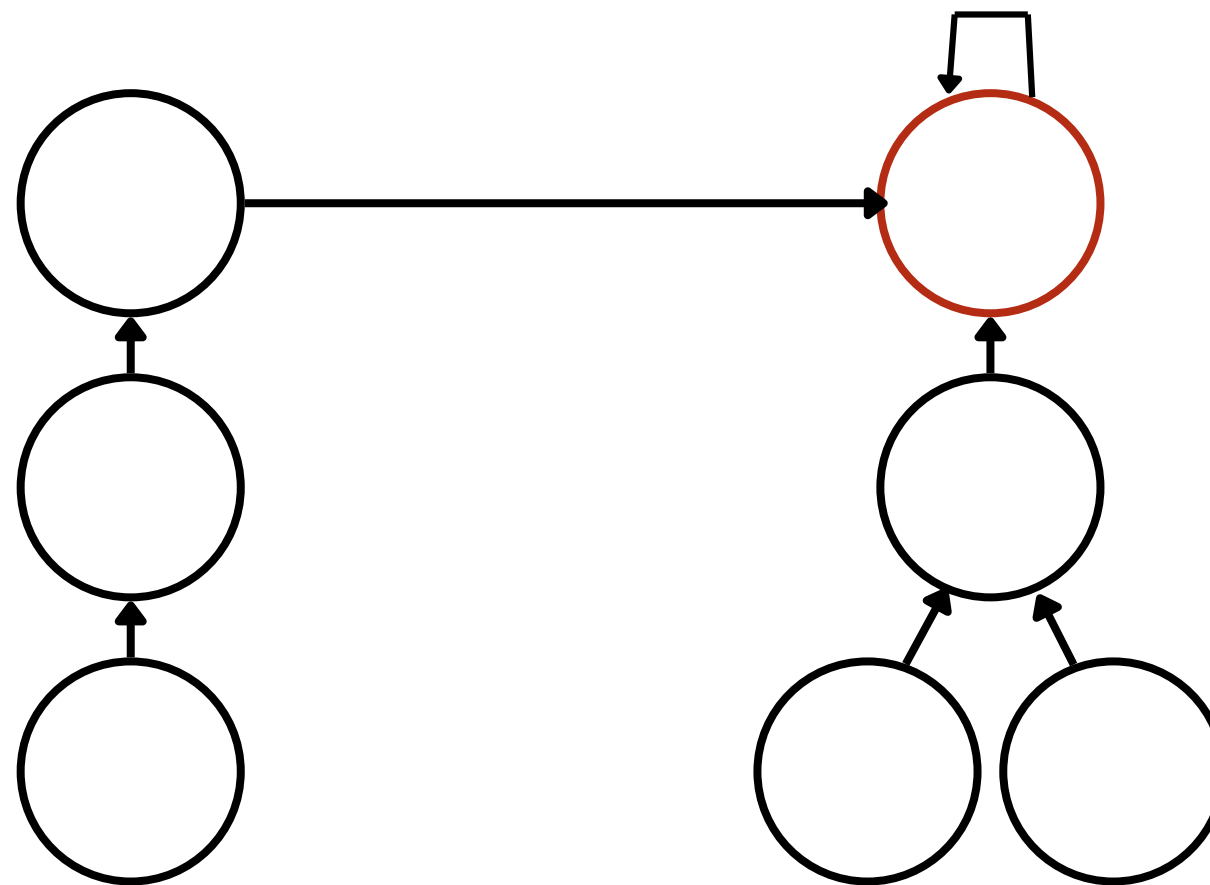


SOLUCIÓN 3.1

Por suerte hay un cambio pequeño pero poderoso que podemos hacer:

Al unir dos líderes, ¿cuál debieramos conectar a cuál?

Si conectamos el más pequeño al más grande, la complejidad será $O(\log N)$



SOLUCIÓN 3.1

```
jefe = [u for u in range(N)]  
size = [1 for u in range(N)]
```

```
def lider(u): ...
```

```
def unir(u, v):  
    u = lider(u)  
    v = lider(v)  
    if u == v: return  
    if size[u] < size[v]: u, v = v, u  
    jefe[v] = u  
    size[u] = size[u] + size[v]
```

SOLUCIÓN 3.2

Podemos optimizar también la función $\text{lider}(u)$.

Si llamamos a $\text{lider}(u)$ muchas veces seguidas, estamos buscando el líder muchas veces innecesariamente.

Intentemos “recordar” este resultado para la próxima vez.

SOLUCIÓN 3.2

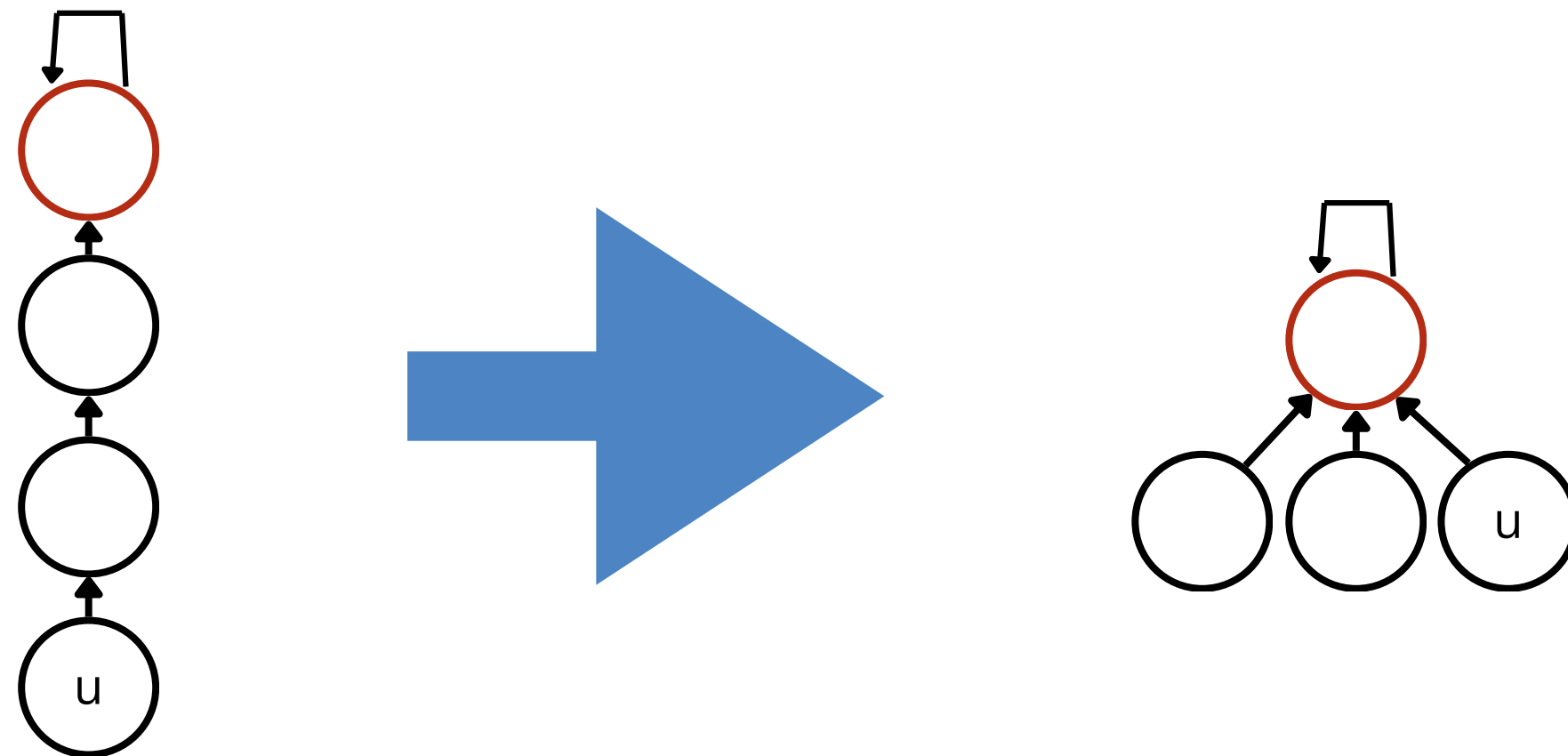
```
jefe = [u for u in range(N)]  
size = [1 for u in range(N)]
```

```
def lider(u):  
    lider_de_u = lider(u)  
    jefe[u] = lider_de_u  
    return lider_de_u
```

```
def unir(u, v): ...
```

SOLUCIÓN 3.2

En pocas palabras, al buscar al líder de **u**, aprovechamos de saltarnos la cadena de mando, y actualizamos el jefe de **u** a su líder.



COMPLEJIDAD 3.2

La complejidad amortizada de ambas operaciones es $O(a(N))$.

Donde $a(N)$ es la *función inversa de Ackermann*.

El análisis está fuera del alcance del curso, pero sabemos lo siguiente:

- La función de Ackermann crece *muy muy rápido*. $\text{Ackermann}(5)$ ya es un número enorme, mayor que 10^{600} .
- Por ende, la función inversa de Ackermann crece *muy muy lento*. $a(N)$ es a lo más 4 para cualquier N razonable (menor que 10^{600}).
- Para nuestros propósitos, $a(N)$ es $O(1)$. Es decir, *en la práctica*, el algoritmo al que llegamos tiene complejidad constante!

RESUMEN

Comenzamos con una estructura de datos simple, con dos operaciones:

- **unir(u, v)**: Unir las componentes conexas de los dos nodos **u** y **v**.
- **lider(u)**: Encontrar el **líder** o nodo-representativo para un nodo **u**.

Optimizamos cada una de estas funciones, y el resultado tiene complejidad casi-constante para ambas operaciones.

A esta estructura de datos le llamamos DSU (disjoint-set-union), o a veces Union-Find (por el nombre de las operaciones en inglés).

PROBLEMA

Tenemos un sistema de N lagunas conectadas por M canales, cada canal i con una profundidad D_i .

Hubo una sequía, por lo que todos los canales se secaron. Sin embargo, un día se pone a diluviar, y los canales se empiezan a llenar poco a poco. En particular, el nivel del agua va subiendo, y va llenando los canales desde el más profundo al menos profundo.

Dado u y v , ¿cuántos canales se alcanzan a llenar antes de poder navegar entre las lagunas u y v ? Encontrar una solución $O(N + M \log M)$.

SOLUCIÓN

Vayamos simulando la situación.

Los canales se llenan de más profundo a menos profundo, así que ordenemos los M canales en este orden. $O(M \log M)$

Inicialicemos también un DSU de tamaño N para llevar cuenta de qué lagunas están conectadas entre sí. $O(N)$

Iteremos por los canales en orden, y vayamos conectando las lagunas.

Después de cada conexión, revisamos si u y v están conectadas. Si lo están, detenemos la iteración, y la cantidad de iteraciones que realizamos es la respuesta que buscamos: la cantidad de canales que se tuvieron que llenar.

PROBLEMA 2

- Tenemos N países. Cada país tiene poder militar A_i .
- Estos países pueden formar alianzas. El aliado de mi aliado también es mi aliado. Es posible que dos países comiencen una guerra solo si no son aliados.
- Nos van llegando consultas para dos países u y v :
 - A veces, se nos informa que u y v formaron una alianza.
 - A veces nos preguntan, si u le declara la guerra a v , ¿ganaría?. u le ganaría la guerra a v si la suma del poder militar de los aliados de u es mas que la suma del poder militar de los aliados de v .

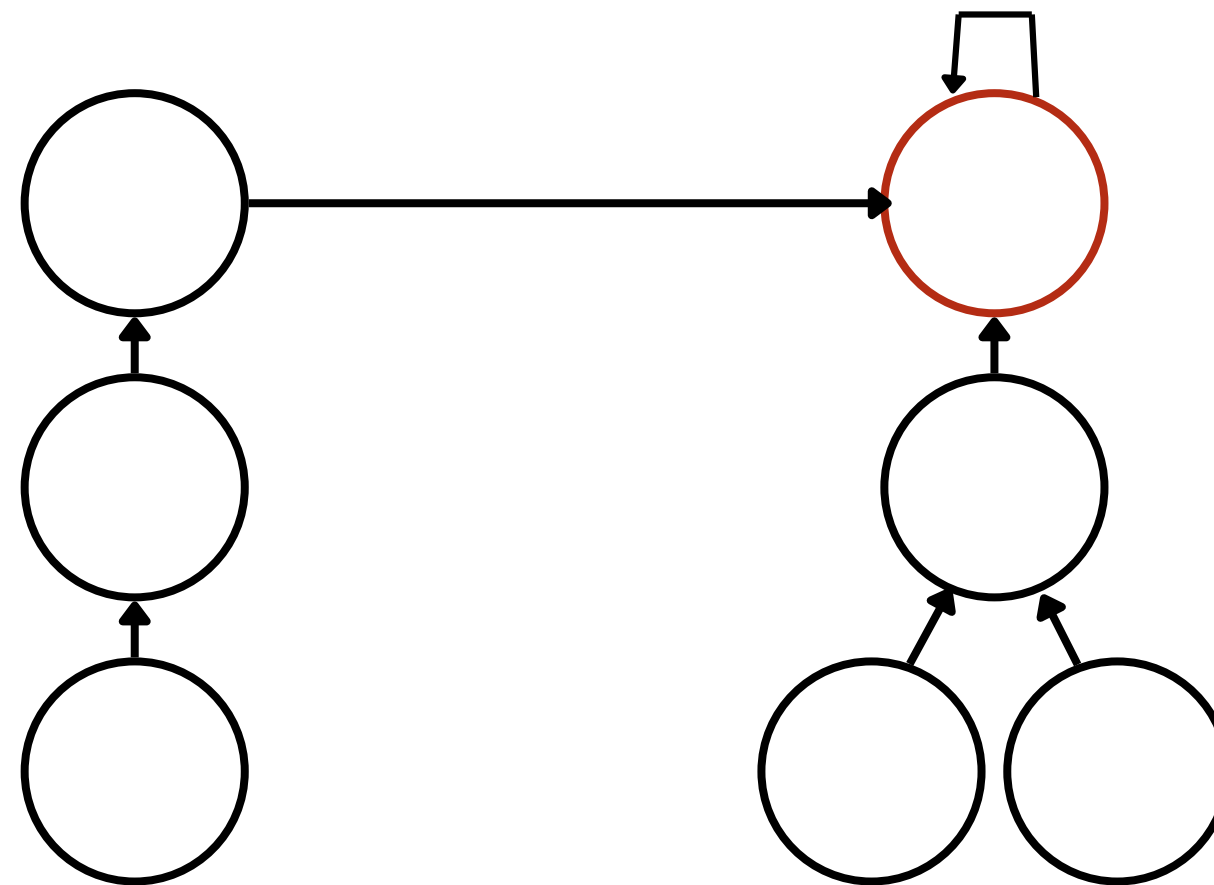
SOLUCIÓN

Al hacer mi DSU, además de guardar el tamaño de la componente, puedo guardar la suma de los poderes militares de la componente. De esta forma puedo saber rápido la suma de los poderes militares de mis aliados, y con esto responder a la consulta.

BONUS: COMPLEJIDAD 3.1

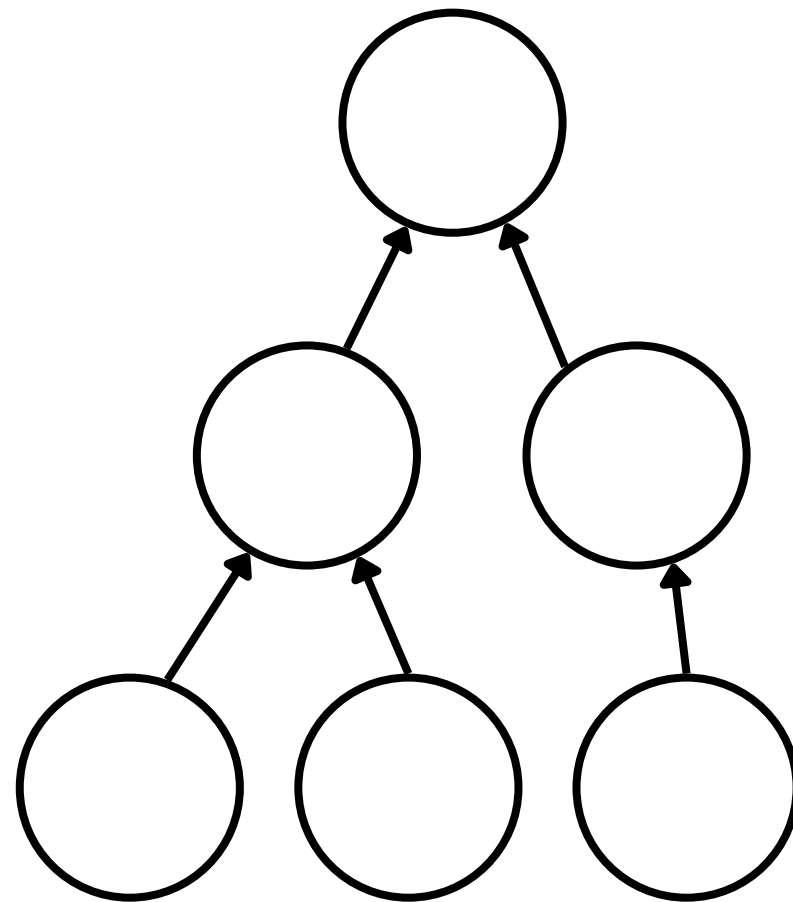
Dijimos que al conectar el grupo más pequeño al grupo más grande, alcanzabamos complejidad $O(\log N)$ para nuestra función líder(u).

¿Por qué?



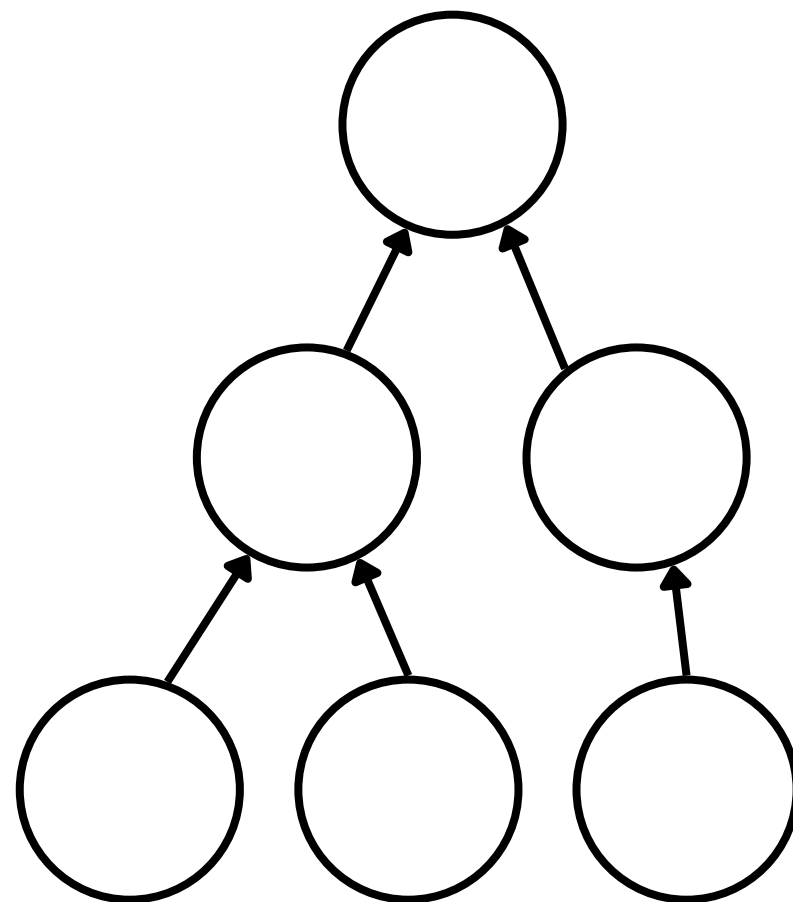
BONUS: COMPLEJIDAD 3.1

Pensemos en una componente conexa de nodos en nuestro grafo de jefes:



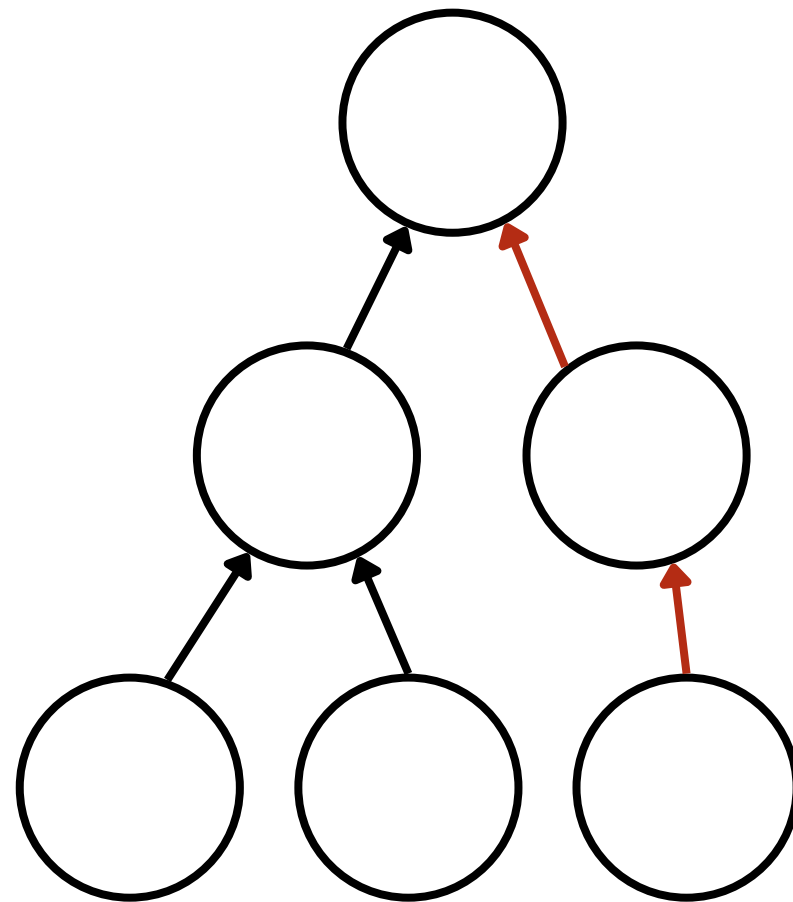
BONUS: COMPLEJIDAD 3.1

Pensemos en una componente conexa de nodos en nuestro grafo de jefes. Este grafo es un **árbol**. Es decir, no hay ciclos, cada nodo tiene exactamente un jefe, cada jefe tiene un conjunto de empleados directos.



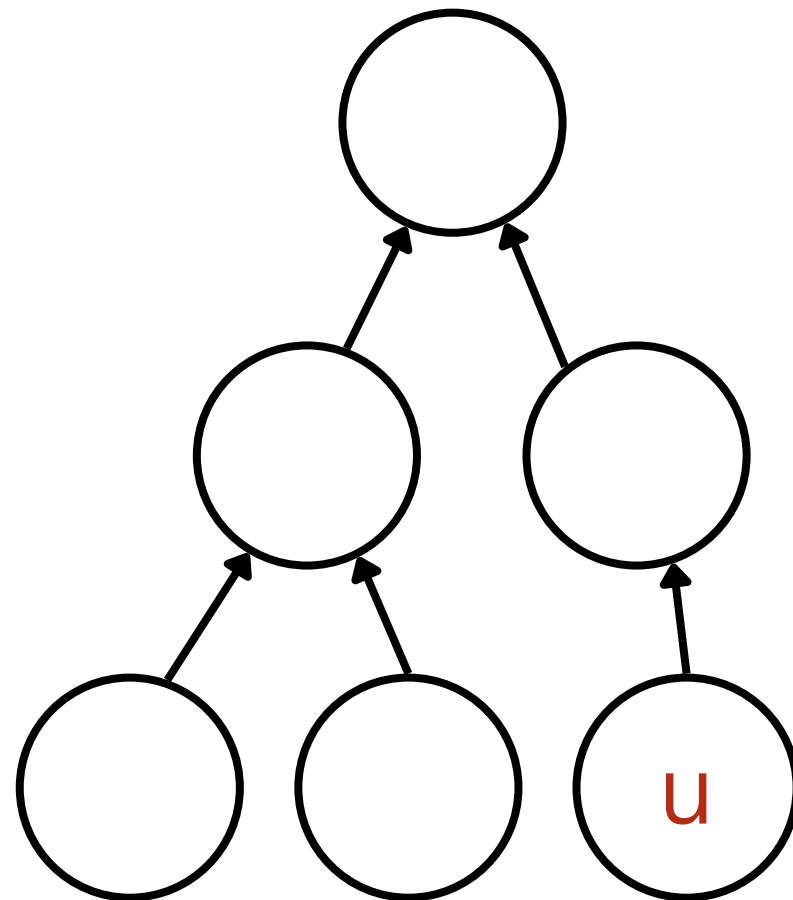
BONUS: COMPLEJIDAD 3.1

La complejidad de nuestra función **lider(u)** es proporcional a que tan **profundo** es este árbol. Es decir, la cantidad de aristas entre el líder y el nodo más que esté más abajo en el árbol.



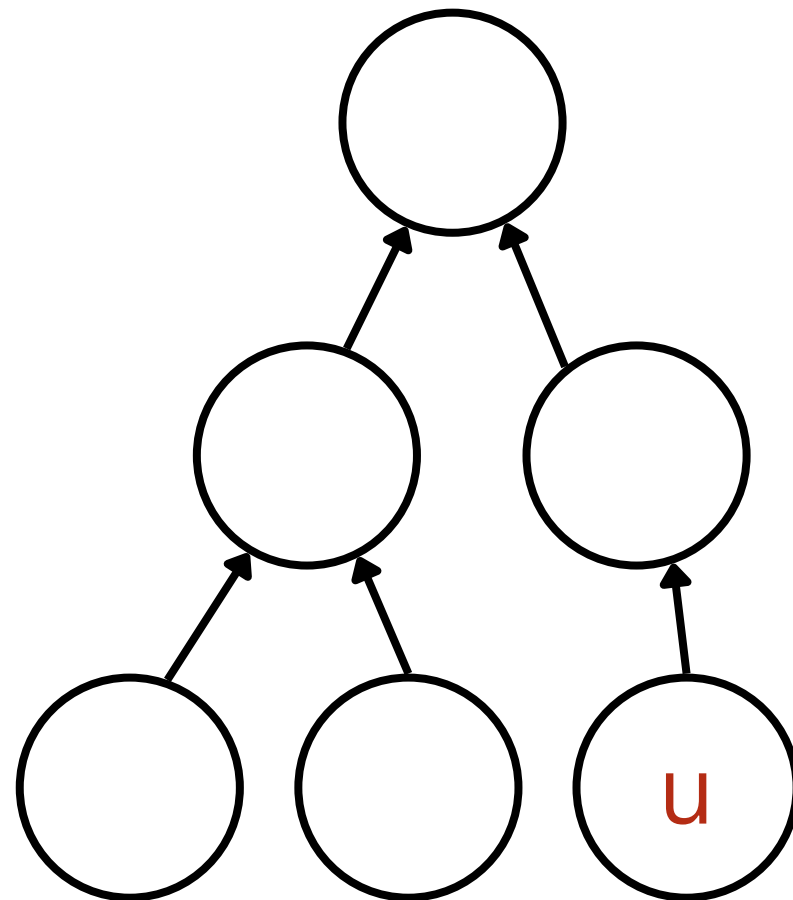
BONUS: COMPLEJIDAD 3.1

Imaginemos que estamos en un nodo en la parte de más abajo del árbol, y queremos llegar hasta el líder. Contemos cuántos “empleados” (nodos bajo nuestro) vamos ganando en cada escalón.



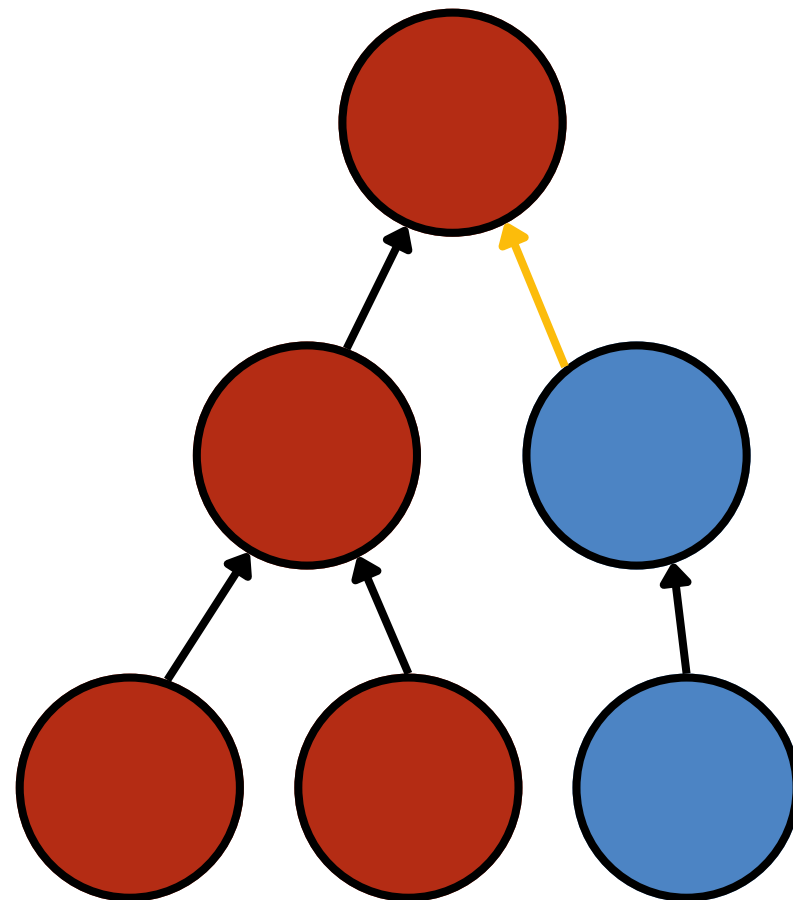
BONUS: COMPLEJIDAD 3.1

Por como construimos las aristas en este árbol, cada arista apunta desde un grupo de nodos a un grupo de nodos **más grande** (o de igual tamaño).



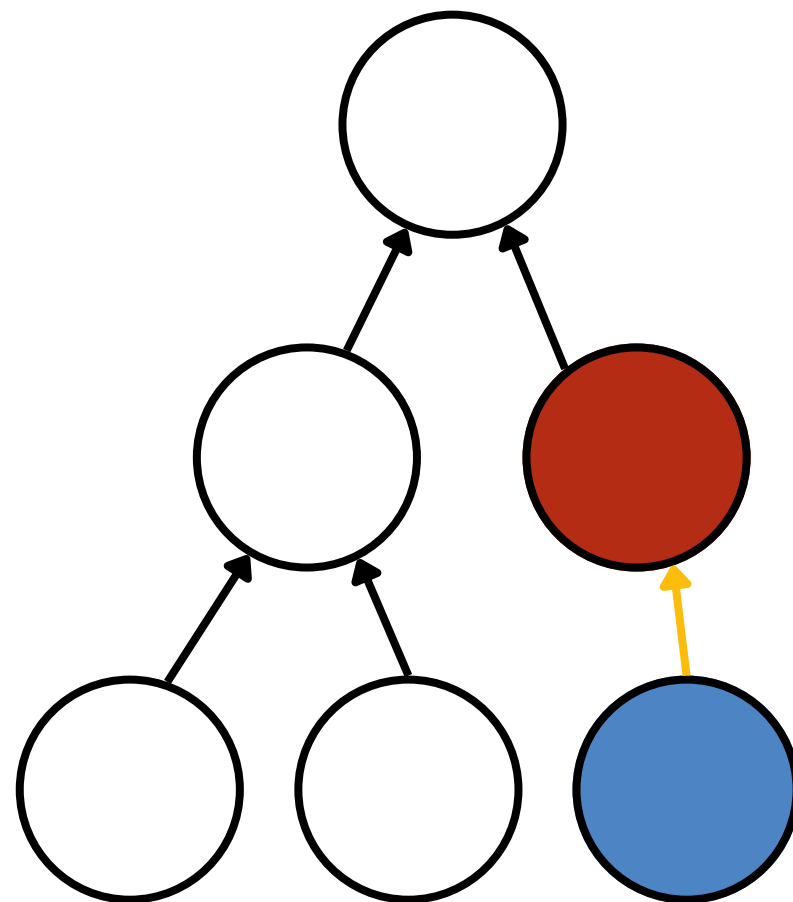
BONUS: COMPLEJIDAD 3.1

Por como construimos las aristas en este árbol, cada arista apunta desde un grupo de nodos a un grupo de nodos **más grande** (o de igual tamaño).



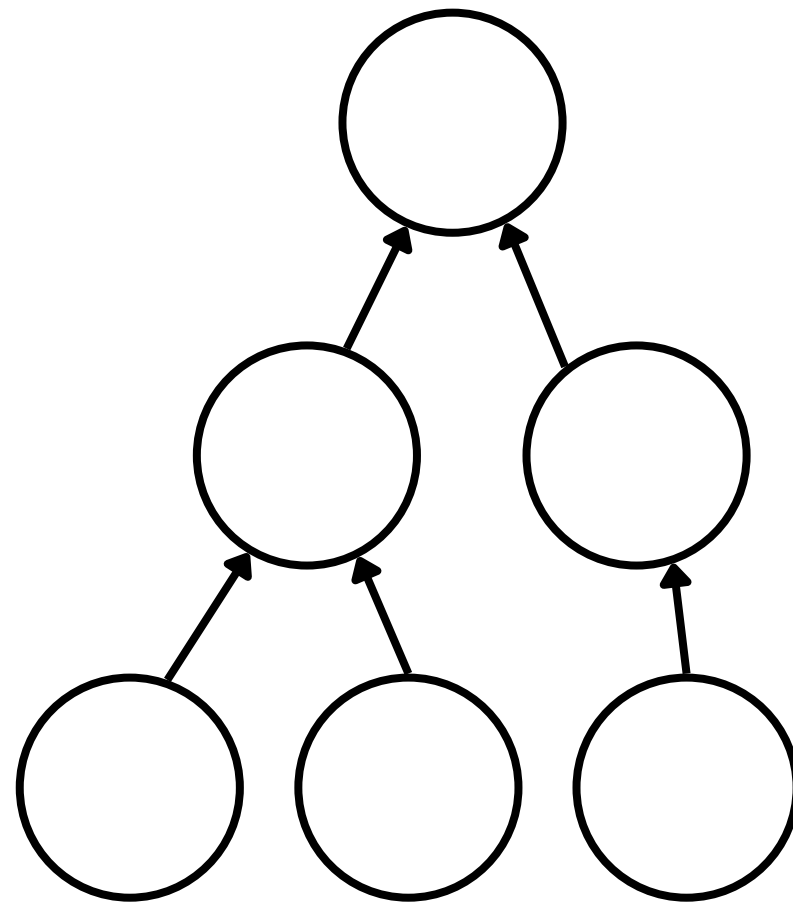
BONUS: COMPLEJIDAD 3.1

Por como construimos las aristas en este árbol, cada arista apunta desde un grupo de nodos a un grupo de nodos **más grande** (o de igual tamaño).



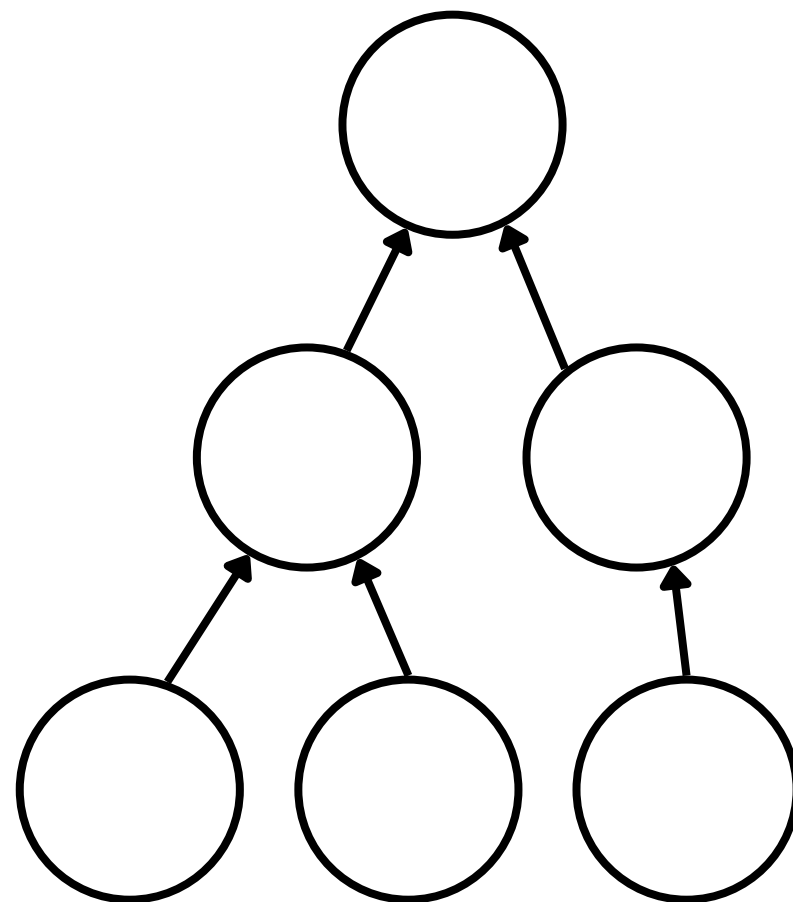
BONUS: COMPLEJIDAD 3.1

Al ir subiendo, cada vez que movamos u a través de una arista, sabemos que estamos “adquiriendo” una cantidad de empleados igual o mayor a la cantidad de empleados que ya teníamos subordinados.



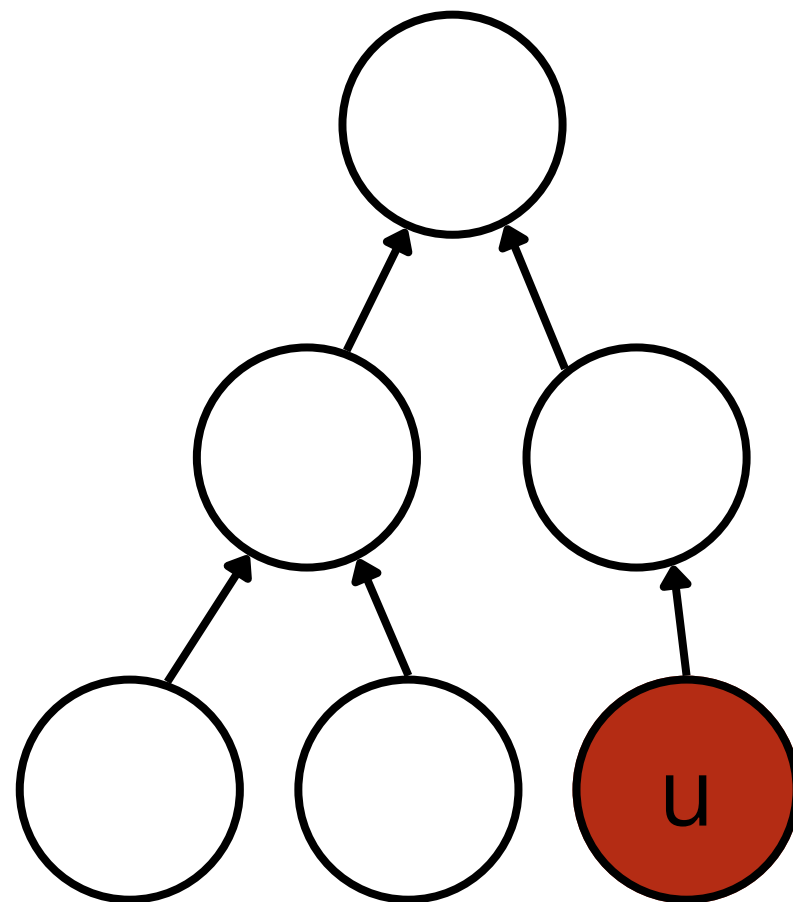
BONUS: COMPLEJIDAD 3.1

Por ende, cada vez que adquirimos una cantidad de empleados igual o mayor que la que ya tenemos, *a lo menos* duplicamos la cantidad total de empleados.



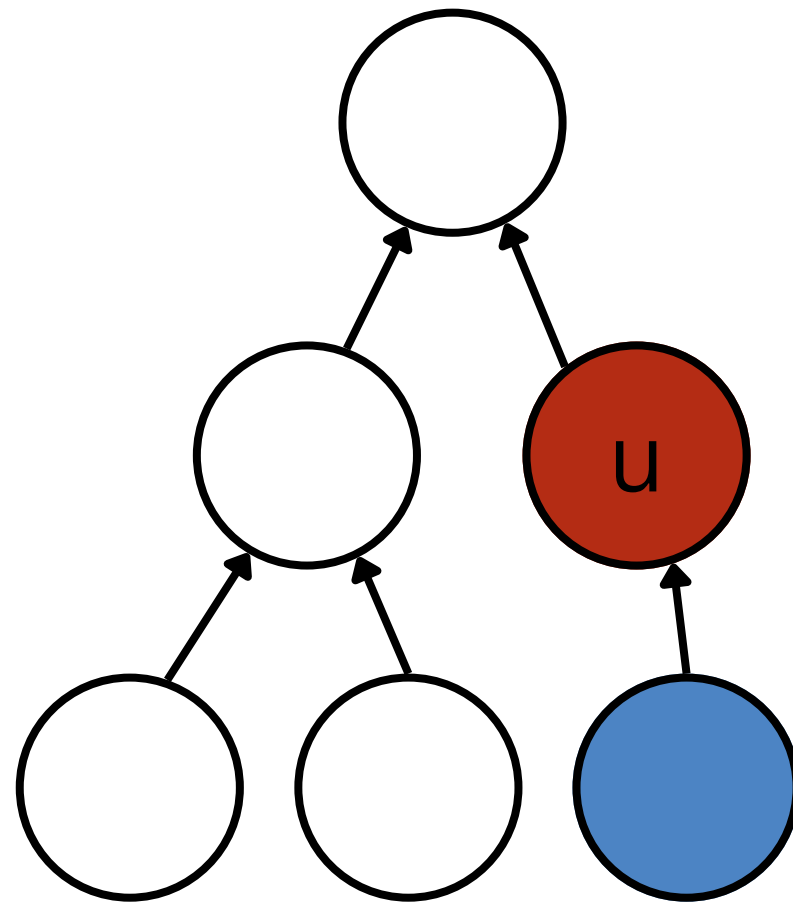
BONUS: COMPLEJIDAD 3.1

Por ende, cada vez que adquirimos una cantidad de empleados igual o mayor que la que ya tenemos, *a lo menos* duplicamos la cantidad total de empleados.



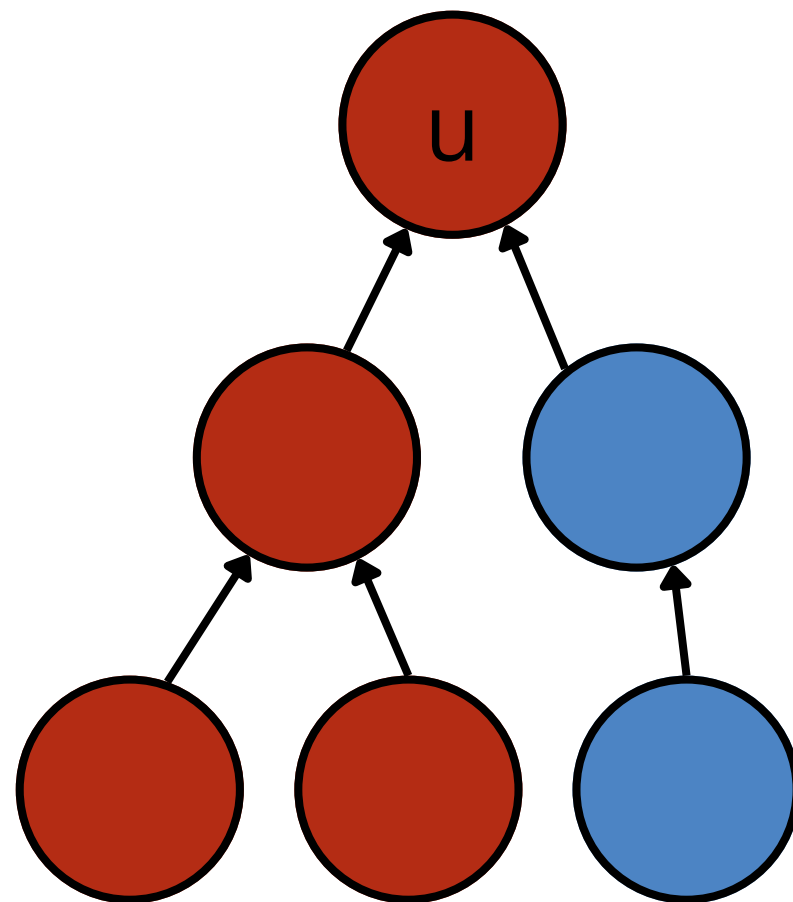
BONUS: COMPLEJIDAD 3.1

Por ende, cada vez que adquirimos una cantidad de empleados igual o mayor que la que ya tenemos, *a lo menos* duplicamos la cantidad total de empleados.



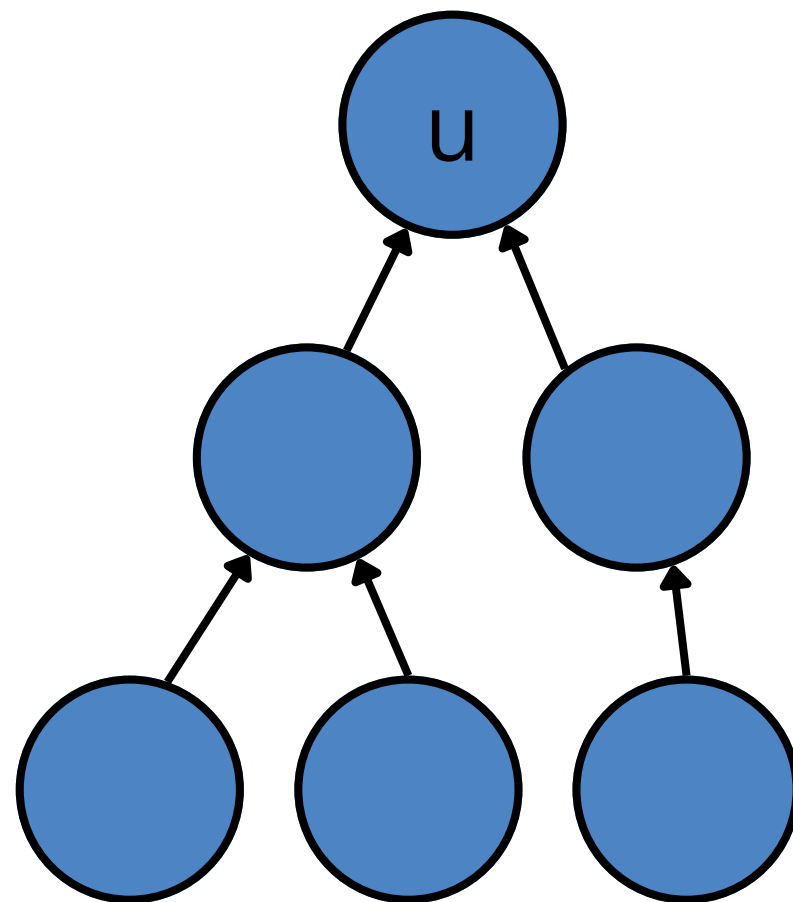
BONUS: COMPLEJIDAD 3.1

Por ende, cada vez que adquirimos una cantidad de empleados igual o mayor que la que ya tenemos, *a lo menos* duplicamos la cantidad total de empleados.



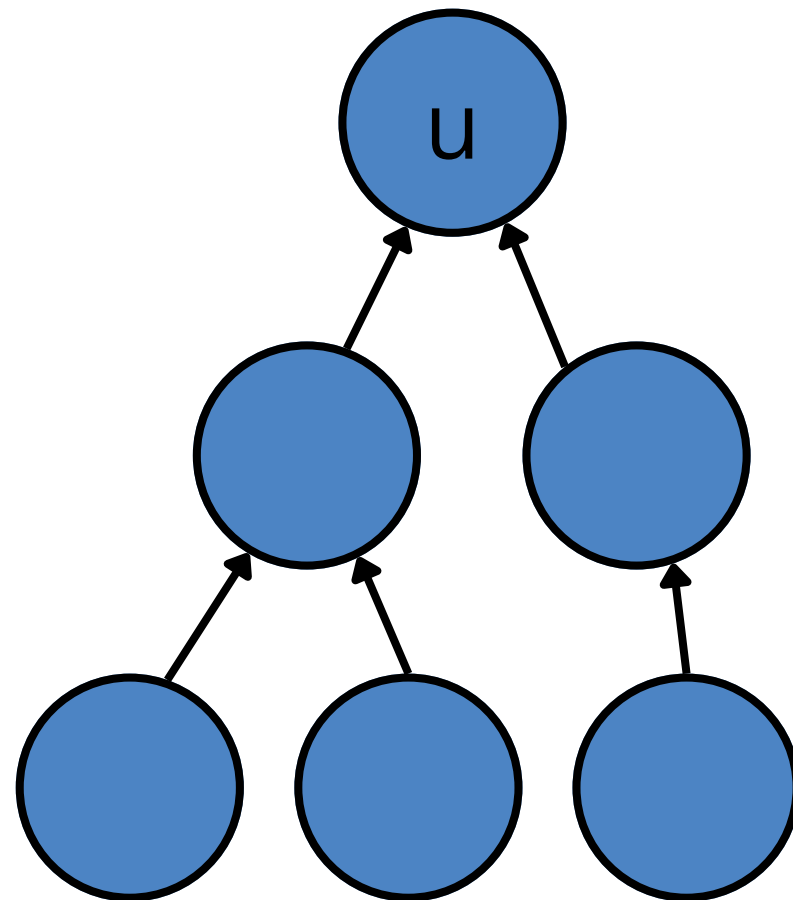
BONUS: COMPLEJIDAD 3.1

Por ende, cada vez que adquirimos una cantidad de empleados igual o mayor que la que ya tenemos, *a lo menos* duplicamos la cantidad total de empleados.



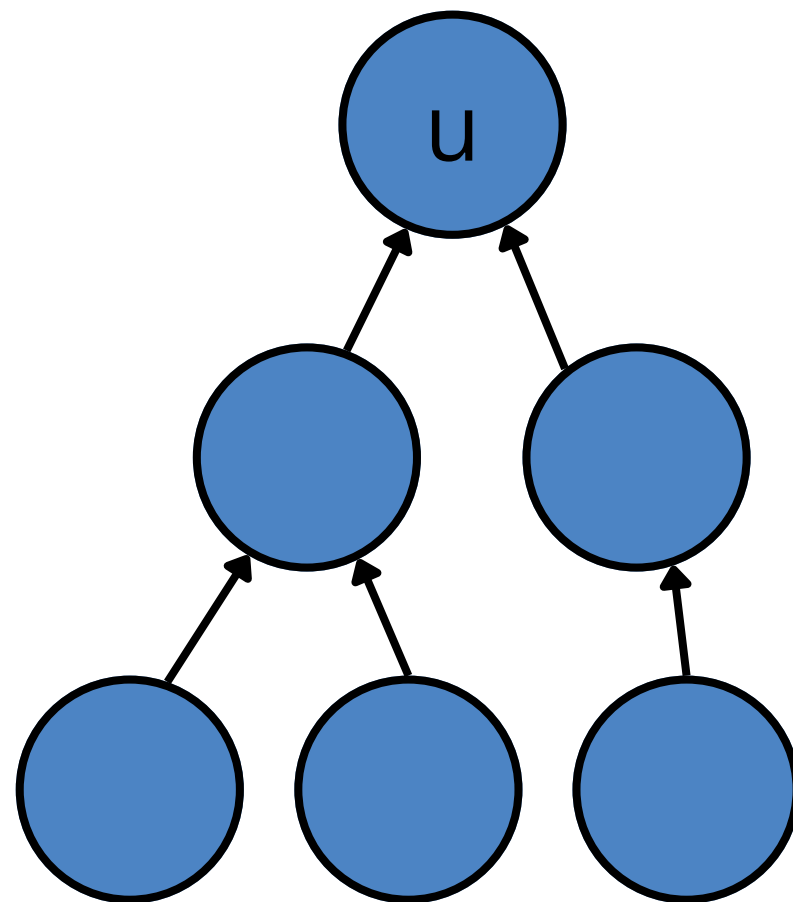
BONUS: COMPLEJIDAD 3.1

Tras trepar k aristas, nuestra cantidad de “empleados” tiene que ser al menos 2^k !



BONUS: COMPLEJIDAD 3.1

Esto significa que si el líder está a **D** aristas del empleado más bajo, el grafo entero tiene a lo menos **2^D** nodos, ya que todos los nodos son “empleados” del líder.

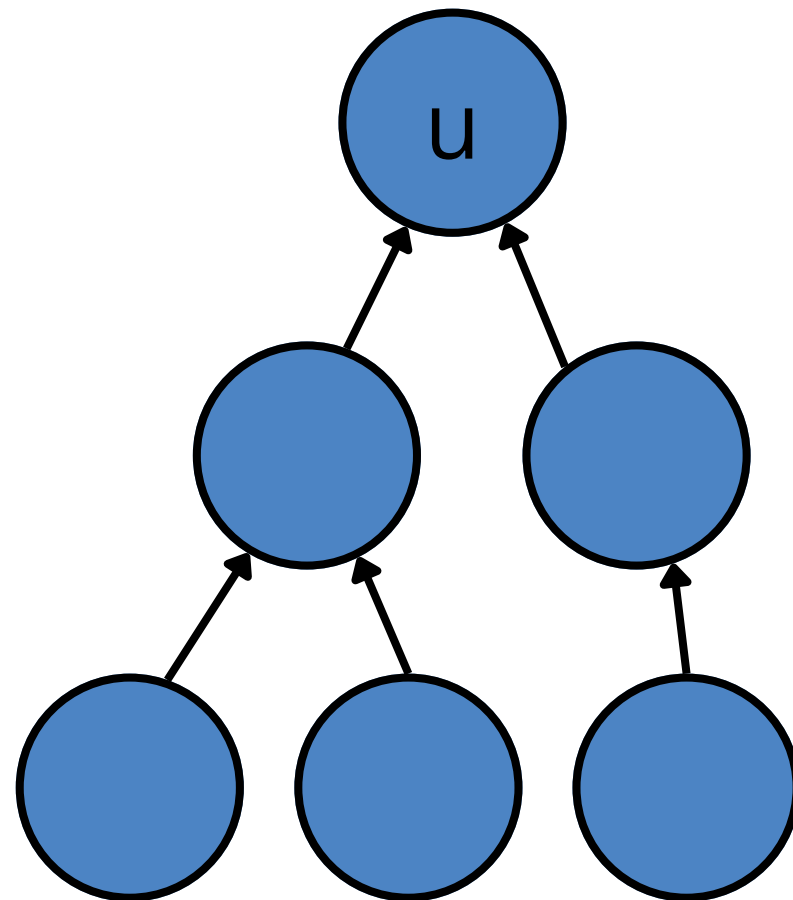


BONUS: COMPLEJIDAD 3.1

Por ende, tenemos una desigualdad entre la profundidad del árbol **D** y la cantidad total de nodos **N**:

$$N \geq 2^D$$

$$\log_2(N) \geq D$$



BONUS: COMPLEJIDAD 3.1

La profundidad del árbol es $O(\log(N))$.

Por ende, nuestra función $\text{lider}(u)$ es $O(\log(N))$.

